



ThunderAgent：一个简单、快速、感知程序的代理推理系统

Hao Kang^{*1}, Ziyang Li^{*2}, Weili Xu^{*3}, Xinyu Yang^{*4}, Yinfang Chen³, Junxiong Wang⁵,
Beidi Chen⁴, Tushar Krishna¹, Chenfeng Xu⁵, and Simran Arora⁵

¹Georgia Institute of Technology

²Individual Researcher

³University of Illinois Urbana-Champaign

⁴Carnegie Mellon University

⁵Together AI

 代码

 博客

 文档

摘要

大型语言模型 (LLM) 已被用于驱动复杂的多轮代理工作流。现有系统松散组合彼此独立的组件来执行代理推理，例如 LLM 推理引擎 vLLM 和工具编排器 Kubernetes。虽然一个代理工作流包含多次 LLM 请求及工具请求，这些系统仍以单个请求为单位分别调度和分配资源，缺乏对整个工作流的端到端了解，因而无法最优地管理 KV 缓存及工具执行环境。为应对这些挑战，我们提出快速、简单、感知程序的代理推理系统 THUNDERAGENT。首先，将代理工作流抽象为 LLM 程序，统一表示异构资源，包括 KV 缓存、系统状态及磁盘空间、网络端口等外部工具资源。在此基础上，THUNDERAGENT 引入程序感知调度器和工具资源管理器，以提高 KV 缓存命中率、缓解内存不均衡，并支持异步环境准备。在编程、路由及科学发现代理上的评估表明，相比先进推理系统，THUNDERAGENT 的服务吞吐量提升为 **1.5–3.6×**，强化学习 rollout 吞吐量提升为 **1.8–3.9×**，磁盘空间占用最多减少 **4.2×**。为便于复现并支持后续开发，我们开源 THUNDERAGENT 的系统实现：<https://github.com/ThunderAgent-org/ThunderAgent>。

1 引言

语言模型的进步使其应用从基础聊天机器人扩展到复杂代理 [21, 27]。这些代理交替进行长篇推理和外部工具调用，例如调用编译器或检索器，以解决编程 [8, 10]、计算机操作 [2, 25] 等领域的真实问题；它们往往自主执行多步骤工作流，无需人实时干预。然而，同时处理的代理请求增多时，现代推理系统的吞吐量会下降 (图 1a)。此外，在强化学习 (RL) 中，rollout 占总实际运行时间的 **70%** 以上 [5, 18]。

随着大规模代理工作流变得更加自主，系统整体效率更多取决于持续吞吐量；有人参与的应用则常由用户响应时间主导。提高吞吐量可将硬件成本分摊到更多完成的工作流，直接降低服务成本。此外，在异步强化学习中，更高的 rollout 吞吐量能减轻采集数据所用参数与正在更新参数之间的策略滞后，使模型从陈旧程度更低的数据中学习，改善收敛速度与最终策略质量 [5, 17, 18, 33]。

然而，现有代理推理系统由独立组件松散组合而成，吞吐量并不理想：现成模型推理引擎，如 vLLM [11] 或 SGLang [34]，与通用工具编排器（例如 Kubernetes）相配合。代理工作流虽包含多轮模型请求和工具请求，各组件仍按单个请求分别调度和分配资源，没有整个工作流的端到端信息。这带来三个关键挑战：

1. **KV 缓存抖动。**高并发导致内存压力时，感知请求的系统为了给其他并发程序的解码请求腾出空间，会被动逐出正在执行工具的程序的 KV 缓存，无法预见工作流后续复用。工具调用完成后，必须重新执行预填充，恢复完整交互历史。再次预填充使工作流平均端到端延迟最多增至 **7.14×** (见 图 1b)，并降低吞吐量。
2. **跨节点内存不均衡。**在多节点推理中，感知请求的引擎存在利用率不均衡。为最大化 KV 缓存命中率，现有引擎将同一代理工作流的所有请求固定到一个节点；但代理上下文会迅速且不可预测地增长，因此这种路由策略使部分节点达到容量上限，其他节点却利用不足。

* 贡献相同。通讯联系：hkang342@gatech.edu

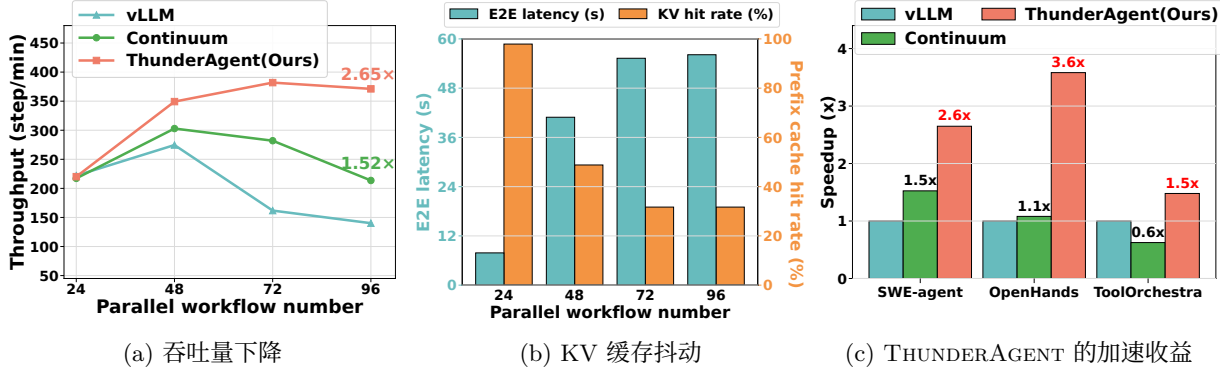


图 1: 并行工作流数（即批大小）增加时，ThunderAgent 与既有代理推理系统的性能比较。在 $8 \times H100$ GPU 集群上，用 GLM-4.6 MoE 模型评估 SWE-Bench Lite 上的 SWE-Agent（图 a、b），以及 SWE-Agent、OpenHands 和 ToolOrchestra（图 c）。结果表明：(a) 现有推理系统在大批大小下无法维持高吞吐量；(b) 吞吐量下降主要由 KV 缓存命中率引起，导致请求端到端延迟增加；(c) THUNDERAGENT 减少 KV 缓存抖动，并管理工具执行资源生命周期，因此获得更高吞吐量。

3. **忽略工具生命周期。**感知请求的编排器难以决定何时释放、何时准备工具执行需要的资源与环境。未使用的沙箱和 API 服务器持续占用关键磁盘空间及网络端口，造成资源逐渐耗尽并引发系统故障。同时，代理工作流开始推理前还需等待极长的环境准备时间。

本文提出 THUNDERAGENT，从端到端视角理解代理工作流，以支持高吞吐代理服务和强化学习 rollout。具体贡献如下：

- 程序抽象：**将代理工作流抽象为代理程序。程序作为一等调度单元，跨多次模型调用和工具执行持续存在，并向运行时暴露语义状态。它跟踪工作流标识符、执行阶段（推理或行动）、调度状态、累计调元数及工具资源等元数据。抽象使调度与执行后端（例如 vLLM、SGLang、TensorRT-LLM）解耦，便于集成新工作流。
- 程序感知调度器：**基于程序抽象，将代理推理调度表述为受 GPU 内存容量约束的优化问题，以尽量降低重计算和缓存开销，最大化预填充与解码吞吐量。我们引入两个关键机制：
 - 状态感知暂停：**执行后端出现内存压力时，选择性暂停处于工具调用行动阶段的工作流，给处于推理阶段的程序保留内存，避免任意且次优的 KV 缓存逐出。
 - 动态迁移：**在数据并行（DP）GPU 节点之间迁移代理程序，以缓解内存不均衡。具体让所有 DP 节点共享一个全局程序感知等待队列，取代始终将同一程序请求发往同一节点的要求。
- 程序感知工具资源管理：**在长期运行的代理负载中，工具环境是持续存在的资源，管理不善会直接限制持续吞吐量。通过跟踪执行依赖，THUNDERAGENT 将 I/O 密集的环境初始化与 LLM 推理重叠。对已完成程序，采用感知生命周期的垃圾回收器，利用程序终止信号回收 Docker 沙箱、网络端口等工具资源，防止资源泄漏累积，使 THUNDERAGENT 持续维持高吞吐量。

仅在感知请求的推理引擎内无法实现上述能力。缺乏程序状态和工作流依赖的显式表示时，调度器不能区分暂时的工具等待与程序终止，也无法协调 GPU 内存和程序级资源调度。

在多类代理负载上评估 THUNDERAGENT。对于**服务**，使用 ToolOrchestra [19] 作为 HLE-Bench [16] 的路由代理，SWE-Agent [28] 和 OpenHands [24] 作为 SWE-bench [10] 的编程代理，并使用 OpenHands 作为 ScienceAgentBench [4] 的科学发现代理。如图 1c 所示，吞吐量提升为 1.48–3.58x。对于**强化学习 rollout**，还在分布式 GPU 节点上测试编程代理，相比先进系统获得 1.79–3.92x 的提升。

此外，在常见大规模部署设置下评估 THUNDERAGENT。如附录所述，最多使用 64 张 H100 GPU 增加工作副本时，吞吐量近似线性扩展（节 A.3）；结合 KV 缓存卸载后仍然有效（节 A.4）。SkyRL 和 NVIDIA Dynamo 等开源框架已采用 THUNDERAGENT（节 A.6）。

2 背景

本节介绍代理推理的特征及现有支持方法。

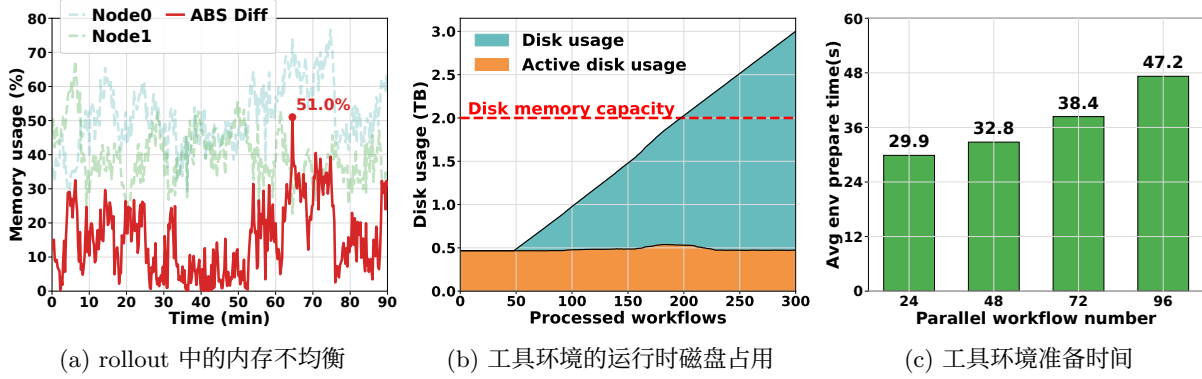


图 2: 当前代理推理系统的内存不均衡及工具资源管理问题。在两个各配 $8 \times H100$ 的 GPU 节点上, 用 vLLM + Kubernetes、GLM 4.6 模型及 SWEBench-Lite 评估 OpenHands 强化学习 rollout。观察表明: (a) 采用 vLLM 的 KV 感知路由时, 90 分钟测试中的最大内存不均衡达 51%; (b) 没有回收工具执行环境, 使资源占用逐渐超过系统容量; (c) 并行工作流数增加时, 工具执行环境的平均准备时间迅速增长。

2.1 当前代理工作流的系统特征

当前代理工作流在生成过程中交替进行推理和行动。形式上, 每个步骤 t 中, 代理接收观测 $o_t \in \mathcal{O}$, 产生输出 $e_t = (l_t, a_t) \in \mathcal{L} \times \mathcal{A}$, 其中 l_t 为思考, a_t 为行动。定义步骤 t 的累计上下文为 $c_t = (o_1, e_1, \dots, o_t)$, 表示工作流的交互历史。以 c_t 为条件, 从策略 $\pi(e_t|c_t)$ 采样 e_t 。

工作流维持两类持久状态: (i) GPU 内存, 其中 c_t 的 KV 缓存充当工作流记忆。随着轨迹增量增长, c_{t+1} 以 c_t 为前缀向后扩展, 因此理论上跨步骤的 KV 缓存复用率接近完全复用。(ii) 工具环境, 例如在 $t=1$ 初始化的沙箱等外部资源, 必须在整个执行期间保持一致、持续可访问。

这些有状态依赖要求从程序级视角理解代理推理轨迹, 使系统能够协调异构资源并管理长期工作流的状态。然而, 现有推理系统将每次思考 l_t 和行动 a_t 视为独立、无状态的请求。

2.2 现有代理推理系统

既有工作主要优化代理推理的单个组件, 例如 LLM 推理引擎或工具编排器 (节 A.1、节 A.2), 跨 GPU、CPU 及远程资源进行工作流端到端优化的研究很少。

Autellix 将多轮代理工作流建模为**仅包含 GPU 的程序**, 在中央进程表中跟踪累计 GPU 执行时间 [14]。但它忽略工作流局部性, 使并发工作流频繁逐出彼此的 KV 缓存, 在重负载下引发 **KV 缓存抖动**。

Continuum 是近期另一个面向多轮代理工作流的服务系统 [12], 利用存活时间 (TTL) 将 KV 缓存固定于 HBM, 缓解工具执行期间的上下文抖动。然而, 它仍无法解决 KV 缓存逐出问题。首先, 多数工具的耗时**不可预测**, 例如 ToolOrchestra [19] 的远程模型 API、编程代理的编译器, 以及计算机操作代理的网页应用 [36]。错误的 TTL 估计会使这些工具在 Continuum 中引发严重抖动及 KV 缓存内存滞留。此外, 运行中工作流的解码内存超过 GPU 上限时, 系统仍会抢占并逐出被固定的 KV 缓存, 导致不可避免的抖动和吞吐量下降, 见图 1a。

这些局限凸显了简单、快速的代理推理系统的必要性。我们设想将其作为新兴代理推理系统的程序感知调度层, 例如 Zhang et al. [32] 的系统。

3 现有代理推理系统的挑战

本节以 vLLM 与 Kubernetes 的组合作为多轮代理推理的代表性基线, 分析其性能并归纳主要低效之处。值得强调的是, 替换推理引擎无法解决这些问题, 需要新的程序感知抽象。默认设置是在两个各配 $8 \times H100$ 的 GPU 节点上, 用 GLM 4.6 生成 OpenHands 强化学习 rollout。

3.1 KV 缓存抖动

代理工作流执行期间, KV 缓存理论复用率很高; 但现有 LLM 服务系统将每个步骤视为独立、无状态的请求。高并发时, 请求级调度在工具执行期间频繁逐出 KV 缓存, 以容纳新到请求, 导致反复逐出和重新预填充; 我们将其称为 **KV 缓存抖动**。

如图 1b 所示，并行工作流增多时抖动加剧。缓存命中率下降，使工具完成后必须重算前缀，带来频繁而昂贵的再次预填充。相比不发生抖动的设置，这种冗余使单个请求的端到端延迟最多增至 7.14 $\times$ ，严重降低吞吐量。

3.2 跨节点内存不均衡

现有跨工作副本路由策略同样不理想。多轮调度器 [22, 34] 为最大化缓存复用，贪心地将请求分配给 KV 缓存局部性最高的目标，却忽略节点之间的内存负载可能失衡。例如，vLLM 的 KV 感知路由器 [22] 将同一代理工作流的所有请求发往同一节点。不同工作流的 KV 占用和生命周期差别很大，因此部分节点过载、部分节点利用很低。类似地，SGLang 的缓存感知路由器使用路由侧基数树来近似工作节点 KV 状态；高代理并发时，工作节点因缓存抖动产生的逐出使这棵树过时，同时导致复用不佳和跨节点不均衡。

如图 2a 所示，在 90 分钟代理强化学习 rollout 的观测中，两个 DP 节点的内存占用差距超过 20% 的时间累计逾 37 分钟，峰值不均衡达 51%。

3.3 忽略工具生命周期

当前代理推理系统没有同步外部工具编排器与 LLM 推理引擎的生命周期，使工具编排侧产生隐性的资源浪费和延迟开销。

资源泄漏与闲置沙箱。 图 2b 显示，总磁盘占用随已处理工作流数线性增加，最终超过系统容量。原因是工作流完成后，没有回收闲置资源，例如已完成负载的 Docker 镜像。低效垃圾回收会给长期运行的代理推理负载带来致命的系统不稳定。

昂贵的环境准备。 我们观察到，多数代理负载开始多轮轨迹前需要准备环境，例如编程代理需要拉取 Docker 镜像、安装相关软件包并构建仓库。准备操作耗时较高，并随并行负载数增加而增长，见图 2c。若 LLM 推理引擎必须等待环境完全就绪，这部分开销会增加推理系统的端到端延迟。

4 ThunderAgent：程序感知的代理推理系统

基于节 3 的分析，我们提出面向高吞吐代理推理的程序感知系统 THUNDERAGENT。节 4.1 建模代理程序，作为调度的核心抽象；节 4.2 形式化用于指导系统设计的成本模型。在此基础上，节 4.3 介绍 KV 缓存调度策略，节 4.4 介绍工具资源管理。

表 1: 代理程序的符号汇总。每个程序实例由其身份、执行阶段、工具环境、资源占用及调度状态刻画。

符号	说明
P	代理程序实例
ID	程序的全局唯一标识符
c	上下文中的词元数
$\mathcal{T}$	程序所需工具环境集合
$\mathcal{L}$	为维持缓存亲和性选择的后端位置
τ	执行阶段：推理（R）或行动（A）
s	调度状态：{Active, Paused, Terminated}（活跃、暂停、终止）

4.1 程序抽象

代理程序是基本抽象，同时封装代理工作流的逻辑执行流程和系统级依赖。形式上，将代理程序 P 定义为元组：

$$P = \langle ID, c, \mathcal{T}, \mathcal{L}, \tau, s \rangle, \quad (1)$$

其中， ID 是全局唯一标识符； c 是上下文词元数，对应活跃执行期间 KV 缓存的内存占用； $\mathcal{T}$ 跟踪程序使用的工具环境集合，以便不再有程序需要时进行垃圾回收。 $\mathcal{L}$ 、 τ 和 s 分别表示节点位置、执行阶段和调度状态，支持程序级 KV 抖动缓解及跨节点转移。图 3 右侧给出一个元数据示例。

THUNDERAGENT 通过 OpenAI 风格端点接口直接封装既有 LLM 引擎和工具编排器。程序 ID 使系统能区分不同工作流的请求。附录 B 进一步说明如何简便地将 THUNDERAGENT 集成到既有推理服务中。

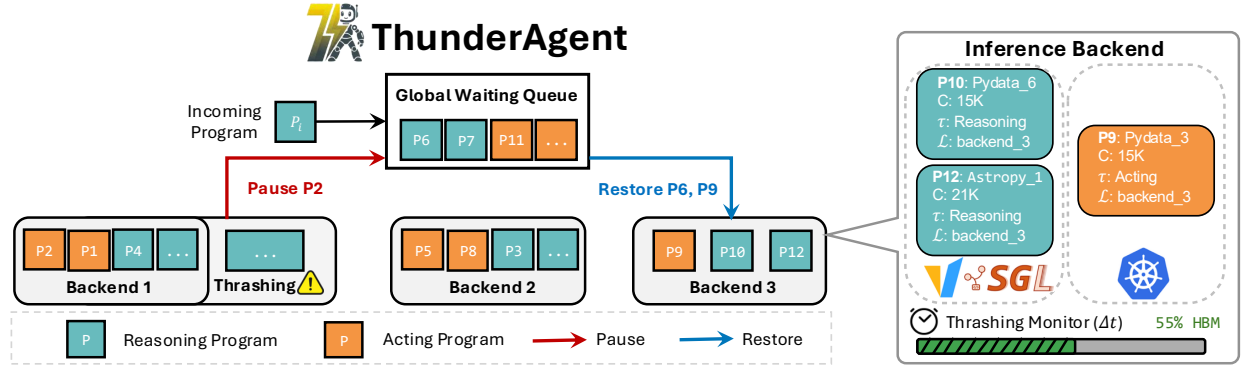


图 3: **ThunderAgent 概览**。展示调度状态转移与内存管理。THUNDERAGENT 每隔 Δt 周期性查询各数据并行后端的状态。图中后端 #1 发生抖动，后端 #3 利用不足。所有后端共享的全局等待队列将行动中的程序 #2 暂停并收回队列，同时放行推理中的程序 #6 和 #9，停止后端 #1 的 KV 缓存抖动，并缓解后端 #3 的内存不均衡。

4.2 成本模型

多轮代理推理中，只有用于实际预填充和解码的资源贡献有效吞吐量，而重计算、容量占用及闲置缓存会造成资源浪费。我们构建 GPU 资源消耗模型，区分有效成本与无产出占用。采用时空乘积（Space-Time Product, STP）[1] 作为主要指标，定义为内存占用对处理时间的积分。一个处理阶段的 STP 成本为：

$$\text{Cost}_x = \int_0^{t_x} M_x(t) dt, \quad (2)$$

其中， t_x 是处理过程 x （例如预填充）的持续时间。LLM 的内存占用 $M_x(t)$ 可直接通过 KV 缓存词元数度量表示，因此成本模型定义为词元数对时间的积分。

代理推理总成本分为五项：解码、预填充、重计算、未使用容量及闲置缓存。我们显式区分工具结果的增量预填充与历史交互的重计算；后者需要在完整上下文上重建被逐出的 KV 缓存，成本显著更高。形式上，总成本分解如下：

$$\text{Cost}_{\text{total}} \approx \text{Cost}_{\text{decode}} + \text{Cost}_{\text{prefill}} + \text{Cost}_{\text{recompute}} + \text{Cost}_{\text{unused}} + \text{Cost}_{\text{caching}} \quad (3)$$

在此分解中， $\text{Cost}_{\text{decode}}$ 和 $\text{Cost}_{\text{prefill}}$ 是贡献推理吞吐量的有效工作，其余是浪费的系统开销： $\text{Cost}_{\text{recompute}}$ 源于 KV 缓存抖动（节 3.1）； $\text{Cost}_{\text{unused}}$ 反映 DP 推理后端副本间的内存不均衡（节 3.2）； $\text{Cost}_{\text{caching}}$ 在外部工具执行期间维持内存时累积（节 3.3）。

4.3 调度策略

根据上述模型，调度目标是最小化无产出的 $\text{Cost}_{\text{recompute}}$ 、 $\text{Cost}_{\text{unused}}$ 和 $\text{Cost}_{\text{caching}}$ ，从而最大化吞吐量。

4.3.1 利用程序感知等待队列减少重计算与缓存成本

如节 3.1 和图 1b 所指出，KV 缓存抖动是吞吐量下降的主要瓶颈。系统必须显式控制活跃程序数，尽量降低 $\text{Cost}_{\text{recompute}}$ 。THUNDERAGENT 为此引入程序感知等待队列，根据程序词元长度 c 和执行阶段 τ ，决定哪些程序在 GPU 上执行，哪些换出。用两个基本操作**恢复 (Restore)**和**暂停 (Pause)**形式化调度器行为：

- **恢复**。将程序接纳到活跃执行中。给定程序 $P = \langle ID, c, \mathcal{T}, \mathcal{L}, \tau, s \rangle$ ，其状态为 $s = \text{Paused}$ 且 $\mathcal{L} = \emptyset$ ， $\text{Restore}(P)$ 将 P 分配到有剩余容量的后端 $\mathcal{L}'$ ，并更新为：

$$P \leftarrow \langle ID, c, \mathcal{T}, \mathcal{L}', \tau, \text{Active} \rangle, \quad (4)$$

- **暂停**。将程序从活跃执行中移除。给定程序 $P = \langle ID, c, \mathcal{T}, \mathcal{L}, \tau, s \rangle$ ，其状态为 $s = \text{Active}$ ， $\text{Pause}(P)$ 解除 P 与后端的绑定，释放其 KV 缓存以供抢占，并更新为：

$$P \leftarrow \langle ID, c, \mathcal{T}, \emptyset, \tau, \text{Paused} \rangle. \quad (5)$$

基于这两个操作，下面介绍尽量减少 KV 缓存抖动的调度策略。

周期性抖动检测。 节 4.1 的程序抽象提供行动阶段程序的 KV 缓存大小，这在请求级系统中不可获取，见节 3。将 DP 后端 $\mathcal{L}$ 的抖动条件定义为程序内存需求超过总容量：

$$C_{\text{total}} < \sum_{p \in \mathcal{L}} c_p \quad (6)$$

其中， C_{total} 为后端 $\mathcal{L}$ 的 KV 缓存池固定词元容量。解码期间，代理上下文长度 c_p 快速增长，即使没有新程序到达，也可能在执行中途触发内存抖动。Continuum 等基线仅在工作流到达时检查是否接纳，而我们实现**周期性监控器**，每隔固定时间 Δt 评估内存占用，提前检测并缓解上下文增长造成的内存压力。

KV 缓存抖动即将发生时，THUNDERAGENT 调用**暂停**操作，挂起活跃程序并释放内存 $\Delta C = \sum_{p \in \mathcal{L}} c_p - \lambda_{\max} \cdot C_{\text{total}}$ ，直到总占用低于 $\lambda_{\max} \cdot C_{\text{total}}$ 。反之，后端有空余空间，即 $\sum_{p \in \mathcal{L}} c_p < \lambda_{\min} \cdot C_{\text{total}}$ 时，通过**恢复**从等待队列接纳暂停程序，并确保恢复后总内存仍低于 $\lambda_{\max} \cdot C_{\text{total}}$ 。 $\lambda_{\max}$ 和 $\lambda_{\min}$ 分别为内存高、低水位，两者共同构成稳定调度的滞回窗口。实践中将二者都设为 1，因为不同程序共享提示词，隐式预留了足够的内存缓冲。

借助程序级周期容量检查，THUNDERAGENT 在行动阶段为活跃程序保留内存，从而保证不出现 KV 缓存抖动；但代价是工具执行很久时，行动程序占用的 GPU 内存处于闲置状态。为权衡缓存与重计算成本，在抖动检查中加入时间衰减机制，逐渐折减行动程序词元的有效权重，使调度器在内存压力升高时逐出长时间闲置的缓存，而不是无限期保留：

$$C_{\text{total}} < \sum_{p \in \mathcal{L}, \tau = \mathbf{R}} c_p + \sum_{q \in \mathcal{L}, \tau = \mathbf{A}} c_q \times f(t_q) \quad (7)$$

具体而言， t_q 是程序 q 当前步骤的工具执行时间， $f(t)$ 是权衡 $\text{Cost}_{\text{caching}}$ 与 $\text{Cost}_{\text{recompute}}$ 的时间衰减函数。衰减后的检查同时控制**暂停**和**恢复**：降低闲置程序权重会允许更多推理程序进入，它们的解码又提升内存压力，触发闲置 KV 缓存逐出。 $f(t)$ 通过随时间降低行动程序的有效内存优先级，促使调度器逐出持续闲置的缓存。在节 F.1 中，我们证明：工具执行延迟满足无记忆性，即剩余执行时间与已用时间独立时，指数衰减是 $f(t)$ 唯一允许的形式。

通过最短优先逐出最小化 $\text{Cost}_{\text{recompute}}$ 确定逐出和恢复条件后，还需选择暂停哪些活跃程序，使重计算成本最低。下面说明，逐出 KV 缓存最小的程序可得到最优解；详细证明见节 F.2。

引理 4.1 (重计算成本的二次增长). 给定上下文长度为 c_i 的程序 P_i ，重新预填充其 KV 缓存的重计算成本随 c_i 二次增长，即：

$$\text{Cost}_{\text{recompute}} = \int_0^{t_{\text{recompute}}} c_i(t) dt \propto c_i^2. \quad (8)$$

定义 4.1 (逐出优化问题). 根据引理 4.1，给定需释放内存 ΔC ，调度器选择逐出程序子集 S ，在释放容量满足约束的同时最小化总重计算成本。优化问题为：

$$\min_S \sum_{i \in S} c_i^2 \quad \text{s.t.} \quad \sum_{i \in S} c_i \geq \Delta C. \quad (9)$$

选取较小的 c_i 严格最小化该目标。因此，THUNDERAGENT 贪心暂停并逐出**上下文最短**的程序。正式证明见附录 F.3。据此分析，恢复与暂停程序采用如下评分：

$$S_{\text{restore}}(P) = \frac{1}{c_P} + \mathbb{I}(\tau = \mathbf{R}) \quad (10)$$

$$S_{\text{pause}}(P) = \frac{1}{c_P} + \mathbb{I}(\tau = \mathbf{A}) \quad (11)$$

其中，指示函数 $\mathbb{I}(\cdot)$ 强制将执行状态 τ 的优先级置于上下文长度之上。两种机制都采用**最短优先**策略以降低重计算成本；但状态指示项 $\mathbb{I}$ 使调度器优先暂停行动程序，通过回收缓存内存最小化 $\text{Cost}_{\text{caching}}$ ，并优先恢复推理程序，以最大化 $\text{Cost}_{\text{decode}} + \text{Cost}_{\text{prefill}}$ 。

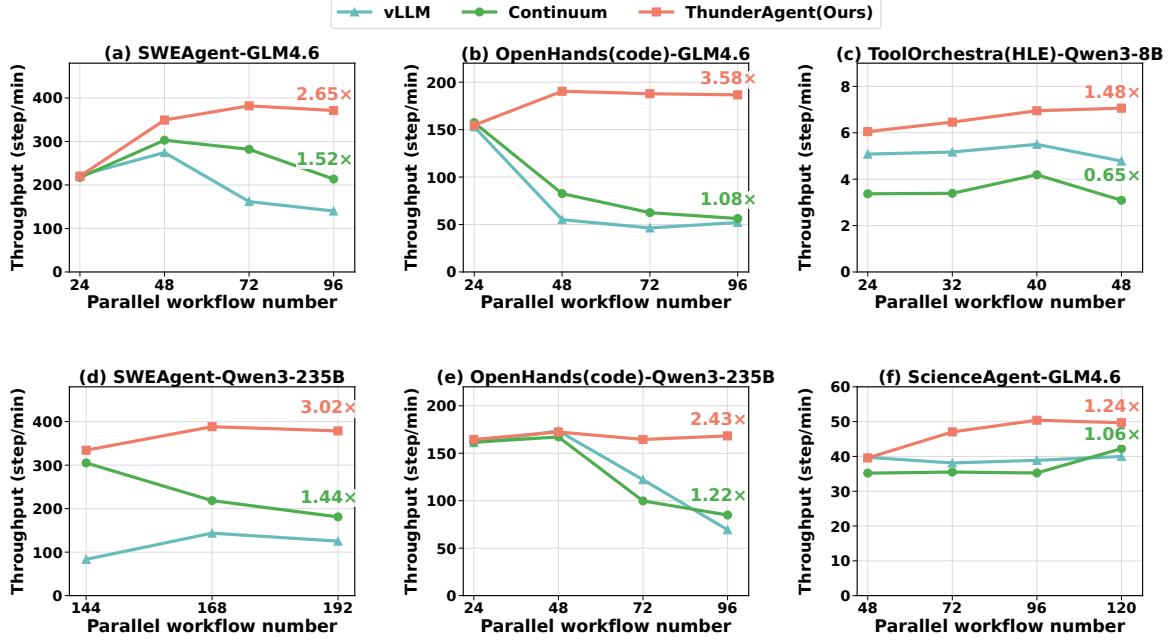


图 4: 服务评估结果。跨三种模型、四类代理工作流和三个数据集, THUNDERAGENT 均优于 vLLM 与 Continuum。工具耗时可预测的工作流 (a、b、d、e) 中, THUNDERAGENT 相对两者最多获得 2.43–3.56 $\times$ 的吞吐量; 工具耗时随机的工作流 (c、f) 中, 吞吐量仍最优。

4.3.2 利用全局程序感知等待队列减少内存不均衡

节 3.1 和 图 2a 表明, 跨节点内存不均衡引入显著 $Cost_{unused}$, 使其他节点容量充足时仍不必要地暂停程序。为此, THUNDERAGENT 将所有后端副本的等待队列统一为**全局程序感知等待队列**。

这一设计的核心动机是: 只有暂停程序仍在等待, 而部分副本有闲置内存时, 才产生 $Cost_{unused}$ 。程序一旦暂停, 就假定其 KV 缓存已被逐出, 因此重计算成本与节点无关, 可以在不牺牲 KV 局部性的情况下改善跨节点均衡。恢复策略采用负载均衡, 而非严格的 KV 感知路由, 允许在有剩余容量的工作副本上恢复程序。因此, 在每个 Δt 周期内, 全局队列使各节点未使用成本满足 $C_{unused} < c_{min} \cdot \Delta t^1$, 其中 c_{min} 为暂停程序中的最小词元长度。图 3 概览了 THUNDERAGENT 的调度策略和全局等待队列。

4.4 工具资源管理

接下来, THUNDERAGENT 缓解 节 3.3 所述的资源泄漏与环境准备开销。

基于钩子的垃圾回收。 实现生命周期钩子, 将工具资源存续与代理程序调度状态 s 严格耦合。程序终止 (*Terminated*) 时, 回收器立即触发拆除流程, 系统性回收沙箱、网络套接字和计算槽位。图 2b 的活跃磁盘占用表明, 这一策略防止过量资源积累, 使磁盘空间消耗随时间保持近乎恒定。

异步环境准备。 初始化工具执行环境, 例如启动 Docker 容器和安装依赖, 可能成为延迟瓶颈。THUNDERAGENT 监控全局等待队列; 高优先级程序, 即 $S_{restore}$ 较高者, 接近恢复阈值时, 在分配 GPU 内存前异步恢复其执行环境。这有效**隐藏初始化开销**, 显著降低编程及科学代理等工具调用密集负载的端到端延迟, 见图 2c。

5 实验

本节在编程、路由、科学研究代理等多类工作流上评估 THUNDERAGENT, 并在从 RTX5090 到 H100 集群的不同硬件配置上评估强化学习 rollout。还通过广泛消融分解端到端运行时间, 并在 节 5.4 说明对调度超

¹由于 Δt 远小于程序生命周期, 忽略一个间隔内程序终止的影响。

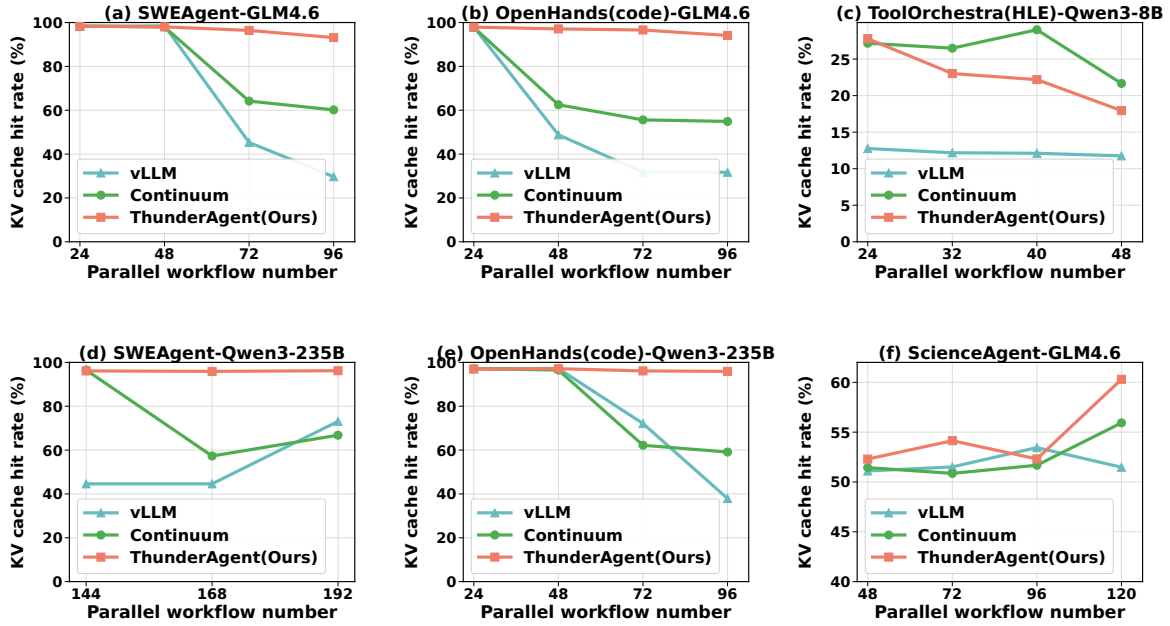


图 5: KV 缓存命中率。工具时长可预测时 (a、b、d、e)，THUNDERAGENT 命中率接近最优；工具时长随机时 (c、f)，动态牺牲部分缓存命中率，以获得更好的计算资源利用。

参数 Δt 与 $f(t)$ 的敏感性。

5.1 实验设置

基准与工作流。 在多类基准和工作负载上评估 THUNDERAGENT：

1. **编程代理服务。** 在 SWEbench-Lite [10] 部署 OpenHands 与 mini-SWEAgent。OpenHands 是**初始化较重**的工作流，每个沙箱平均磁盘占用超过 10GB；mini-SWEAgent 是**轻量**工作流，每个沙箱只占约 2GB。
2. **通用代理服务。** 在 HLE [16] 使用 ToolOrchestra，在 ScienceAgentBench [4] 使用 OpenHands；延迟随外部 API 调用及复杂科学仿真变化。
3. **强化学习 rollout。** 在两个各配 $8 \times H100$ 的节点上，采用相同模型、工作流和样本生成 rollout。

模型与部署。 用 GLM-4.6 (355B) [21] 和 Qwen-3 (235B) [27] 分别运行 OpenHands [24] 和 mini-SWEAgent [28]。模型量化为 FP8，在 $8 \times H100$ 节点上采用八路张量并行 (TP8)。ToolOrchestra [19] 使用 FP16 的 Qwen3-8B，部署于一张 RTX 5090。LLM 引擎和 Docker 分属不同集群：引擎运行于承载模型的 GPU 集群，代理 Docker 环境卸载至专用 CPU 集群。

ThunderAgent 配置。 按 节 4.3 的定义，将超参数设为 $\Delta t = 5$ ，优先级衰减设为 $f(t) = 2^{-t}$ 。使用 vLLM 为 LLM 推理引擎；吞吐量以每分钟步骤数度量，一个步骤包括一段推理和一段行动。

基线方法。 比较采用不同调度范式的先进系统：

- **vLLM (推理)：** 广泛采用的请求感知 LLM 引擎，作为无状态的推理性能基线，不包含代理或程序特定信息。
- **Continuum (推理)：** 当前先进的多轮代理工作流系统，通过预测工具时长并据此将 KV 缓存固定在 HBM 中，缓解缓存抖动。
- **vLLM + SGLang Gateway (分布式 rollout)：** 大规模分布式强化学习 rollout 的领先方案。SGLang Gateway 改善跨节点内存均衡及 KV 缓存命中率，构成这一设置下较强的基线。

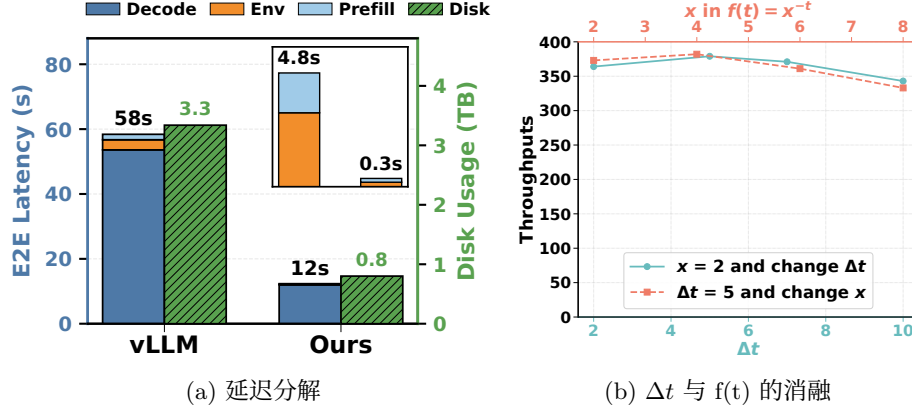


图 6: THUNDERAGENT 的端到端延迟分解及参数敏感性消融研究。

5.2 服务评估结果

高并发下的高吞吐量。 图 4 显示，高并发（例如 96 个并行程序）时，THUNDERAGENT 在不同基础模型和数据集上，相比 vLLM 加速 **1.48–3.58×**，相比 Continuum 加速 **1.17–3.31×**。收益来自程序感知调度器维持接近最优的 KV 缓存命中率：Mini-SWE-Bench 与 OpenHands 约为 $\approx 100\%$ ，见图 5 的 a、b、d、e；同时支持异步准备环境。相比之下，Continuum 在高并发下退化；如图 5 所示，缓存命中率从 $>90\%$ 显著下降至 $\approx 60\%$ 。这是因为运行中请求解码内存不足时，不同程序互相逐出 KV 缓存，活跃程序竞争有限内存并引发抖动。

高并发下的稳健性。 即使并行工作流数增长到超过 GPU 内存上限，THUNDERAGENT 仍维持可达到的最大吞吐量。图 4 显示，工作流数增加时，本文系统吞吐量保持稳定，而基线一旦超过内存限制便严重下降。由于代理环境及工具时长具有随机性，实践中很难预先确定一个既充分利用资源、又只产生有限 KV 抖动和缓存成本的最佳并行数。THUNDERAGENT 自动适应可用容量，无需人工调参；这对稳健的真实部署很重要。

确定性与随机工具执行下的稳健性。 THUNDERAGENT 在工具模式确定的工作流（图 4 的 a、b、d、e）及高度随机的条件（图 4 的 c、f）下，均优于基线。原因是动态程序感知等待队列策略。vLLM 的请求感知调度器通常不为行动程序保留内存，导致频繁重计算；Continuum 则静态为暂停程序预留内存，并可能误判工具时长。因此，长且不可预测的工具调用会造成昂贵的 $\text{Cost}_{\text{recompute}}$ 或 $\text{Cost}_{\text{caching}}$ 。THUNDERAGENT 通过时间衰减 $f(t)$ 平衡两者：优先为工具耗时短的程序保留缓存，提前暂停工具耗时长长的程序，避免浪费内存。如图 5 右侧所示，随机场景中，THUNDERAGENT 缓存命中率虽低于 Continuum，但通过维持 GPU 的有效利用，获得更高吞吐量。

5.3 rollout 评估结果

表 2: 在 $2 \times \text{H100}$ 节点上生成 GLM-4.6 rollout ($N = 144$)。

工作流	服务系统	吞吐量
mini-SWEAgent	vLLM + Gateway	375.4
mini-SWEAgent	THUNDERAGENT	671.8 (1.79×)
OpenHands	vLLM + Gateway	69.1
OpenHands	THUNDERAGENT	270.8 (3.92×)

在双节点 H100 集群上，用 GLM-4.6 评估强化学习 rollout。表 2 显示，THUNDERAGENT 维持有效扩展能力，相比 vLLM + Gateway，吞吐量提升为 **1.79–3.92**×，适合内存密集的分布式强化学习负载。

5.4 消融研究

端到端延迟分解。 图 6a 分解 OpenHands rollout 的平均端到端延迟。吞吐量收益主要来自**预填充和解码延迟**减少。此外，工具资源管理策略（节 4.4）贡献约 10% 的延迟改善，并使磁盘空间占用减少 4.2×。每步端到端延迟在附录 E 进一步讨论。

Δt 与 $f(t)$ 的消融。 研究检测周期 Δt 和衰减函数 $f(t) = x^{-t}$ 的敏感性。图 6b 在一个 H100 节点上，以 GLM4.6 为基础模型离线服务 mini-SWEAgent。不同参数设置下，THUNDERAGENT 均保持高吞吐量，说明方法稳健。进一步增大 Δt 可能使两次检测之间发生抖动，降低缓存命中率与吞吐量。增大 $f(t)$ 中的 x 会更积极地逐出行动程序，以增加重计算成本换取较少缓存成本；工具执行较短的行动程序被过早逐出时，吞吐量因而下降。

6 结论

我们提出快速、简单的代理系统 THUNDERAGENT，以程序级抽象跟踪每个代理 workflow 整个生命周期中的元数据，并利用这一抽象进行运行时调度和资源管理。具体而言，THUNDERAGENT 动态调度程序在 GPU 节点上的执行，缓解 KV 缓存抖动和内存不均衡，同时管理工具资源以防止资源泄漏。实验表明，相比既有系统，THUNDERAGENT 的服务吞吐量提升为 **1.48–3.58**×，强化学习 rollout 吞吐量提升为 **1.79–3.92**×。

致谢

感谢 Together AI 使本工作得以开展。感谢 Ben Athiwaratkun 和 Ce Zhang 协助开发多后端调度器；感谢 Wenyi Hong 和 Luke Huang 在研究期间提供有益反馈与讨论。

影响说明

本研究旨在推动机器学习系统领域的发展，具体目标是优化代理工作流的执行效率。通过显著降低维持代理推理及强化学习 rollout 所需的内存和硬件资源，本文方法改善大规模代理服务的**成本效率**和**能源可持续性**。它使资源有限的硬件也能够训练和推理更大模型，并运行更复杂的环境。这种效率改善有助于企业及个人开展代理研究，让更多机构和实践者无需承担过高计算投入，即可参与高保真仿真。虽然提高系统吞吐量理论上可能加快自主代理的有益和有害应用，但本文主要面向基础设施优化，这是本领域可扩展、可持续发展的基础。

参考文献

- [1] L. A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966. doi: 10.1147/sj.52.0078.
- [2] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Buckner, Lawrence Jang, and Zack Hui. Windows agent arena: Evaluating multi-modal os agents at scale, 2024. URL <https://arxiv.org/abs/2409.08264>.
- [3] Shiyi Cao, Dacheng Li, Fangzhou Zhao, Shuo Yuan, Sumanth R Hegde, Connor Chen, Charlie Ruan, Tyler Griggs, Shu Liu, Eric Tang, et al. Skyr-agent: Efficient rl training for multi-turn llm agent. *arXiv preprint arXiv:2511.16108*, 2025.
- [4] Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. Scienceagentbench: Toward rigorous assessment of language agents for data-driven scientific discovery, 2024. URL <https://arxiv.org/abs/2410.05080>.
- [5] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jia-shu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. Areal: A large-scale asynchronous reinforcement learning system for language reasoning, 2025. URL <https://arxiv.org/abs/2505.24298>.
- [6] Wei Gao, Yuheng Zhao, Tianyuan Wu, Shaopan Xiong, Weixun Wang, Dakai An, Lunxi Cao, Dilxat Muhtar, Zichen Liu, Haizhou Zhao, et al. Rollart: Scaling agentic rl training via disaggregated infrastructure. *arXiv preprint arXiv:2512.22560*, 2025.
- [7] Yiyuan He, Minoxian Xu, Jingfeng Wu, Jianmin Hu, Chong Ma, Min Shen, Le Chen, Chengzhong Xu, Lin Qu, and Kejiang Ye. Banaserve: Unified kv cache and dynamic module migration for balancing disaggregated llm serving in ai infrastructure, 2025. URL <https://arxiv.org/abs/2510.13223>.
- [8] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL <https://arxiv.org/abs/2403.07974>.
- [9] Dongfu Jiang, Yi Lu, Zhuofeng Li, Zhiheng Lyu, Ping Nie, Haozhe Wang, Alex Su, Hui Chen, Kai Zou, Chao Du, et al. Verltool: Towards holistic agentic reinforcement learning with tool use. *arXiv preprint arXiv:2509.01055*, 2025.
- [10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- [11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. URL <https://arxiv.org/abs/2309.06180>.
- [12] Hanchen Li, Qiuyang Mang, Runyuan He, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. Continuum: Efficient and robust multi-turn llm agent scheduling with kv cache time-to-live, 2025. URL <https://arxiv.org/abs/2511.02230>.
- [13] Yuhao Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. Lmcache: An efficient kv cache layer for enterprise-scale llm inference, 2025. URL <https://arxiv.org/abs/2510.09665>.
- [14] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. Autellix: An efficient serving engine for llm agents as general programs, 2025. URL <https://arxiv.org/abs/2502.13965>.

- [15] Emanuel Parzen. *Stochastic processes*. SIAM, 1999.
- [16] Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, Michael Choi, Anish Agrawal, Arnav Chopra, Adam Khoja, Ryan Kim, Richard Ren, Jason Hausenloy, Oliver Zhang, Mantas Mazeika, Dmitry Dodonov, Tung Nguyen, Jaeho Lee, Daron Anderson, Mikhail Doroshenko, Alun Cennyth Stokes, Mobeen Mahmood, Oleksandr Pokutnyi, Oleg Iskra, Jessica P. Wang, John-Clark Levin, Mstyslav Kazakov, Fiona Feng, Steven Y. Feng, Haoran Zhao, Michael Yu, Varun Gangal, Chelsea Zou, Zihan Wang, Serguei Popov, Robert Gerbicz, Geoff Galgon, Johannes Schmitt, Will Yeadon, Yongki Lee, Scott Sauers, Alvaro Sanchez, Fabian Giska, Marc Roth, Søren Riis, Saiteja Utpala, Noah Burns, Gashaw M. Goshu, Mohinder Maheshbhai Naiya, Chidozie Agu, Zachary Giboney, Antrell Cheatom, Francesco Fournier-Facio, Sarah-Jane Crowson, Lennart Finke, Zerui Cheng, Jennifer Zampese, Ryan G. Hoerr, Mark Nandor, Hyunwoo Park, Tim Gehringer, Jiaqi Cai, Ben McCarty, Alexis C Garretson, Edwin Taylor, Damien Sileo, Qiuyu Ren, Usman Qazi, Lianghui Li, Jungbae Nam, John B. Wydallis, Pavel Arkhipov, Jack Wei Lun Shi, Aras Bacho, Chris G. Willcocks, Hangrui Cao, Sumeet Motwani, Emily de Oliveira Santos, Johannes Veith, Edward Vendrow, Doru Cojoc, Kengo Zenitani, Joshua Robinson, Longke Tang, Yuqi Li, Joshua Vendrow, Natanael Wildner Fraga, Vladyslav Kuchkin, Andrey Pupasov Maksimov, Pierre Marion, Denis Efremov, Jayson Lynch, Kaiqu Liang, Aleksandar Mikov, Andrew Gritsevskiy, Julien Guillod, Gözdenur Demir, Dakotah Martinez, Ben Pageler, Kevin Zhou, Saeed Soori, Ori Press, Henry Tang, Paolo Rissone, Sean R. Green, Lina Brüssel, Moon Twayana, Aymeric Dieuleveut, Joseph Marvin Imperial, Ameya Prabhu, Jinzhou Yang, Nick Crispino, Arun Rao, Dimitri Zvonkine, Gabriel Loiseau, Mikhail Kalinin, Marco Lukas, Ciprian Manolescu, Nate Stambaugh, Subrata Mishra, Tad Hogg, Carlo Bosio, Brian P Coppola, Julian Salazar, Jaehyeok Jin, Rafael Sayous, Stefan Ivanov, Philippe Schwaller, Shaipranesh Senthilkuma, Andres M Bran, Andres Algaba, Kelsey Van den Houte, Lynn Van Der Sypt, Brecht Verbeken, David Noever, Alexei Kopylov, Benjamin Myklebust, Bikun Li, Lisa Schut, Evgenii Zheltonozhskii, Qiaochu Yuan, Derek Lim, Richard Stanley, Tong Yang, John Maar, Julian Wykowski, Martí Oller, Anmol Sahu, Cesare Giulio Ardito, Yuzheng Hu, Ariel Ghislain Kemogne Kamdoun, Alvin Jin, Tobias Garcia Vilchis, Yuexuan Zu, Martin Lackner, James Koppel, Gongbo Sun, Daniil S. Antonenko, Steffi Chern, Bingchen Zhao, Pierrot Arsene, Joseph M Cavanagh, Daofeng Li, Jiawei Shen, Donato Crisostomi, Wenjin Zhang, Ali Dehghan, Sergey Ivanov, David Perrella, Nurdin Kaparov, Allen Zang, Ilia Sucholutsky, Arina Kharlamova, Daniil Orel, Vladislav Poritski, Shalev Ben-David, Zachary Berger, Parker Whitfill, Michael Foster, Daniel Munro, Linh Ho, Shankar Sivarajan, Dan Bar Hava, Aleksey Kuchkin, David Holmes, Alexandra Rodriguez-Romero, Frank Sommerhage, Anji Zhang, Richard Moat, Keith Schneider, Zakayo Kazibwe, Don Clarke, Dae Hyun Kim, Felipe Meneguitti Dias, Sara Fish, Veit Elser, Tobias Kreiman, Victor Efren Guadarrama Vilchis, Immo Klose, Ujjwala Anantheswaran, Adam Zweiger, Kaivalya Rawal, Jeffery Li, Jeremy Nguyen, Nicolas Daans, Haline Heidinger, Maksim Radionov, Václav Rozhoň, Vincent Ginis, Christian Stump, Niv Cohen, Rafał Poświata, Josef Tkadlec, Alan Goldfarb, Chenguang Wang, Piotr Padlewski, Stanislaw Barzowski, Kyle Montgomery, Ryan Stendall, Jamie Tucker-Foltz, Jack Stade, T. Ryan Rogers, Tom Goertzen, Declan Grabb, Abhishek Shukla, Alan Givré, John Arnold Ambay, Archan Sen, Muhammad Fayez Aziz, Mark H Inlow, Hao He, Ling Zhang, Younesse Kaddar, Ivar Ångquist, Yanxu Chen, Harrison K Wang, Kalyan Ramakrishnan, Elliott Thornley, Antonio Terpin, Hailey Schoelkopf, Eric Zheng, Avishy Carmi, Ethan D. L. Brown, Kelin Zhu, Max Bartolo, Richard Wheeler, Martin Stehberger, Peter Bradshaw, JP Heimonen, Kaustubh Sridhar, Ido Akov, Jennifer Sandlin, Yury Makarychev, Joanna Tam, Hieu Hoang, David M. Cunningham, Vladimir Goryachev, Demosthenes Patramanis, Michael Krause, Andrew Redenti, David Aldous, Jesyin Lai, Shannon Coleman, Jiangnan Xu, Sangwon Lee, Ilias Magoulas, Sandy Zhao, Ning Tang, Michael K. Cohen, Orr Paradise, Jan Hendrik Kirchner, Maksym Ovchinnikov, Jason O. Matos, Adithya Shenoy, Michael Wang, Yuzhou Nie, Anna Sztyber-Betley, Paolo Faraboschi, Robin Riblet, Jonathan Crozier, Shiv Halasyamani, Shreyas Verma, Prashant Joshi, Eli Meril, Ziqiao Ma, Jérémy Andréoletti, Raghav Singhal, Jacob Platnick, Volodymyr Nevirkovets, Luke Basler, Alexander Ivanov, Seri Khoury, Nils Gustafsson, Marco Piccardo, Hamid Mostaghimi, Qijia Chen, Virendra Singh, Tran Quoc Khánh, Paul Rosu, Hannah Szlyk, Zachary Brown, Himanshu Narayan, Aline Menezes, Jonathan Roberts, William Alley, Kunyang Sun, Arkil Patel, Max Lamparth, Anka Reuel, Linwei Xin, Hanmeng Xu, Jacob Loader, Freddie Martin, Zixuan Wang, Andrea Achilleos, Thomas Preu, Tomek Korbak, Ida Bosio, Fereshteh

Kazemi, Ziyue Chen, Biró Bálint, Eve J. Y. Lo, Jiaqi Wang, Maria Inês S. Nunes, Jeremiah Milbauer, M Saiful Bari, Zihao Wang, Behzad Ansarinejad, Yewen Sun, Stephane Durand, Hossam Elgnainy, Guillaume Douville, Daniel Tordera, George Balabanian, Hew Wolff, Lynna Kvistad, Hsiaoyun Milliron, Ahmad Sakor, Murat Eron, Andrew Favre D. O., Shailesh Shah, Xiaoxiang Zhou, Firuz Kamalov, Sherwin Abdoli, Tim Santens, Shaul Barkan, Allison Tee, Robin Zhang, Alessandro Tomasiello, G. Bruno De Luca, Shi-Zhuo Looi, Vinh-Kha Le, Noam Kolt, Jiayi Pan, Emma Rodman, Jacob Drori, Carl J Fossum, Niklas Muennighoff, Milind Jagota, Ronak Pradeep, Honglu Fan, Jonathan Eicher, Michael Chen, Kushal Thaman, William Merrill, Moritz Firsching, Carter Harris, Stefan Ciobăcă, Jason Gross, Rohan Pandey, Ilya Gusev, Adam Jones, Shashank Agnihotri, Pavel Zhelnov, Mohammadreza Mofayezi, Alexander Piperski, David K. Zhang, Kostiantyn Dobarskyi, Roman Leventov, Ignat Soroko, Joshua Duersch, Vage Taamazyan, Andrew Ho, Wenjie Ma, William Held, Ruicheng Xian, Arnel Randy Zebaze, Mohanad Mohamed, Julian Noah Leser, Michelle X Yuan, Laila Yacar, Johannes Lengler, Katarzyna Olszewska, Claudio Di Fratta, Edson Oliveira, Joseph W. Jackson, Andy Zou, Muthu Chidambaram, Timothy Manik, Hector Haffenden, Dashiell Stander, Ali Dasouqi, Alexander Shen, Bitá Golshani, David Stap, Egor Kretov, Mikalai Uzhou, Alina Borisovna Zhidkovskaya, Nick Winter, Miguel Orbegozo Rodriguez, Robert Lauff, Dustin Wehr, Colin Tang, Zaki Hossain, Shaun Phillips, Fortuna Samuele, Fredrik Ekström, Angela Hammon, Oam Patel, Faraz Farhidi, George Medley, Forough Mohammadzadeh, Madellene Peñaflor, Haile Kassahun, Alena Friedrich, Rayner Hernandez Perez, Daniel Pyda, Taom Sakal, Omkar Dhamane, Ali Khajegili Mirabadi, Eric Hallman, Kenchi Okutsu, Mike Battaglia, Mohammad Maghsoudimehrabani, Alon Amit, Dave Hulbert, Roberto Pereira, Simon Weber, Handoko, Anton Peristy, Stephen Malina, Mustafa Mehkary, Rami Aly, Frank Reidegeld, Anna-Katharina Dick, Cary Friday, Mukhwinder Singh, Hassan Shapourian, Wanyoung Kim, Mariana Costa, Hubeyb Gurdogan, Harsh Kumar, Chiara Ceconello, Chao Zhuang, Haon Park, Micah Carroll, Andrew R. Tawfeek, Stefan Steinerberger, Daattavya Aggarwal, Michael Kirchhof, Linjie Dai, Evan Kim, Johan Ferret, Jainam Shah, Yuzhou Wang, Minghao Yan, Krzysztof Burdzy, Lixin Zhang, Antonio Franca, Diana T. Pham, Kang Yong Loh, Joshua Robinson, Abram Jackson, Paolo Giordano, Philipp Petersen, Adrian Cosma, Jesus Colino, Colin White, Jacob Votava, Vladimir Vinnikov, Ethan Delaney, Petr Spelda, Vit Stritecky, Syed M. Shahid, Jean-Christophe Mourrat, Lavr Vetoshkin, Koen Sponselee, Renas Bacho, Zheng-Xin Yong, Florencia de la Rosa, Nathan Cho, Xiuyu Li, Guillaume Malod, Orion Weller, Guglielmo Albani, Leon Lang, Julien Laurendeau, Dmitry Kazakov, Fatimah Adesanya, Julien Portier, Lawrence Hollom, Victor Souza, Yuchen Anna Zhou, Julien Degorre, Yiğit Yalın, Gbenga Daniel Obikoya, Rai, Filippo Bigi, M. C. Boscá, Oleg Shumar, Kaniuar Bacho, Gabriel Recchia, Mara Popescu, Nikita Shulga, Ngefor Mildred Tanwie, Thomas C. H. Lux, Ben Rank, Colin Ni, Matthew Brooks, Alesia Yakimchyk, Huanxu, Liu, Stefano Cavalleri, Olle Häggström, Emil Verkama, Joshua Newbould, Hans Gundlach, Leonor Brito-Santana, Brian Amaro, Vivek Vajipey, Rynaa Grover, Ting Wang, Yosi Kratish, Wen-Ding Li, Sivakanth Gopi, Andrea Caciolai, Christian Schroeder de Witt, Pablo Hernández-Cámara, Emanuele Rodolà, Jules Robins, Dominic Williamson, Vincent Cheng, Brad Raynor, Hao Qi, Ben Segev, Jingxuan Fan, Sarah Martinson, Erik Y. Wang, Kaylie Hausknecht, Michael P. Brenner, Mao Mao, Christoph Demian, Peyman Kassani, Xinyu Zhang, David Avagian, Eshawn Jessica Scipio, Alon Ragoler, Justin Tan, Blake Sims, Rebeka Plecnik, Aaron Kirtland, Omer Faruk Bodur, D. P. Shinde, Yan Carlos Leyva Labrador, Zahra Adoul, Mohamed Zekry, Ali Karakoc, Tania C. B. Santos, Samir Shamseldeen, Loukmane Karim, Anna Liakhovitskaia, Nate Resman, Nicholas Farina, Juan Carlos Gonzalez, Gabe Maayan, Earth Anderson, Rodrigo De Oliveira Pena, Elizabeth Kelley, Hodjat Mariji, Rasoul Pouriamanesh, Wentao Wu, Ross Finocchio, Ismail Alarab, Joshua Cole, Danyelle Ferreira, Bryan Johnson, Mohammad Safdari, Liangti Dai, Siriphan Arthornthurasuk, Isaac C. McAlister, Alejandro José Moyano, Alexey Pronin, Jing Fan, Angel Ramirez-Trinidad, Yana Malysheva, Daphiny Pottmaier, Omid Taheri, Stanley Stepanic, Samuel Perry, Luke Askew, Raúl Adrián Huerta Rodríguez, Ali M. R. Minissi, Ricardo Lorena, Krishnamurthy Iyer, Arshad Anil Fasiludeen, Ronald Clark, Josh Ducey, Matheus Piza, Maja Somrak, Eric Vergo, Juehang Qin, Benjámín Borbás, Eric Chu, Jack Lindsey, Antoine Jallon, I. M. J. McInnis, Evan Chen, Avi Semler, Luk Gloor, Tej Shah, Marc Carauleanu, Pascal Lauer, Tran Duc Huy, Hossein Shahrtash, Emilien Duc, Lukas Lewark, Assaf Brown, Samuel Albanie, Brian Weber, Warren S. Vaz, Pierre Clavier, Yiyang Fan, Gabriel Poesia Reis e Silva, Long, Lian, Marcus Abramovitch, Xi Jiang, Sandra Mendoza, Murat Islam, Juan Gonzalez, Vasilios Mavroudis, Justin Xu, Pawan Kumar, Laxman Prasad Goswami, Daniel Bugas, Nasser Heydari, Ferenc Jeanp-

long, Thorben Jansen, Antonella Pinto, Archimedes Apronti, Abdallah Galal, Ng Ze-An, Ankit Singh, Tong Jiang, Joan of Arc Xavier, Kanu Priya Agarwal, Mohammed Berkani, Gang Zhang, Zhehang Du, Benedito Alves de Oliveira Junior, Dmitry Malishev, Nicolas Remy, Taylor D. Hartman, Tim Tarver, Stephen Mensah, Gautier Abou Loume, Wiktor Morak, Farzad Habibi, Sarah Hoback, Will Cai, Javier Gimenez, Roselynn Grace Montecillo, Jakub Łucki, Russell Campbell, Asankhaya Sharma, Khalida Meer, Shreen Gul, Daniel Espinosa Gonzalez, Xavier Alapont, Alex Hoover, Gunjan Chhablani, Freddie Vargus, Arunim Agarwal, Yibo Jiang, Deepakkumar Patil, David Outevsky, Kevin Joseph Scaria, Rajat Maheshwari, Abdelkader Dendane, Priti Shukla, Ashley Cartwright, Sergei Bogdanov, Niels Mündler, Sören Möller, Luca Arnaboldi, Kunvar Thaman, Muhammad Rehan Siddiqi, Prajvi Saxena, Himan-shu Gupta, Tony Fruhauff, Glen Sherman, Mátyás Vincze, Siranut Usawasutsakorn, Dylan Ler, Anil Radhakrishnan, Innocent Enyekwe, Sk Md Salaudhin, Jiang Muzhen, Aleksandr Maksapetyan, Vivien Rossbach, Chris Harjadi, Mohsen Bahaloohoreh, Claire Sparrow, Jasdeep Sidhu, Sam Ali, Song Bian, John Lai, Eric Singer, Justine Leon Uro, Greg Bateman, Mohamed Sayed, Ahmed Menshawy, Darling Duclosel, Dario Bezzi, Yashaswini Jain, Ashley Aaron, Murat Tiryakioglu, Sheeshram Siddh, Keith Krennek, Imad Ali Shah, Jun Jin, Scott Creighton, Denis Peskoff, Zienab EL-Wasif, Ragavendran P V, Michael Richmond, Joseph McGowan, Tejal Patwardhan, Hao-Yu Sun, Ting Sun, Nikola Zubić, Samuele Sala, Stephen Ebert, Jean Kaddour, Manuel Schottdorf, Dianzhuo Wang, Gerol Petruzella, Alex Meiburg, Tilen Medved, Ali ElSheikh, S Ashwin Hebbar, Lorenzo Vaquero, Xianjun Yang, Jason Poulos, Vilém Zouhar, Sergey Bogdanik, Mingfang Zhang, Jorge Sanz-Ros, David Anugraha, Yinwei Dai, Anh N. Nhu, Xue Wang, Ali Anil Demircali, Zhibai Jia, Yuyin Zhou, Juncheng Wu, Mike He, Nitin Chandok, Aarush Sinha, Gaoxiang Luo, Long Le, Mickaël Noyé, Michał Perelkiewicz, Ioannis Pantidis, Tianbo Qi, Soham Sachin Purohit, Letitia Parcalabescu, Thai-Hoa Nguyen, Genta Indra Winata, Edoardo M. Ponti, Hanchen Li, Kaustubh Dhole, Jongee Park, Dario Abbondanza, Yuanli Wang, Anupam Nayak, Diogo M. Caetano, Antonio A. W. L. Wong, Maria del Rio-Chanona, Dániel Kondor, Pieter Francois, Ed Chalstrey, Jakob Zsambok, Dan Hoyer, Jenny Reddish, Jakob Hauser, Francisco-Javier Rodrigo-Ginés, Suchandra Datta, Maxwell Shepherd, Thom Kamphuis, Qizheng Zhang, Hyunjun Kim, Ruiji Sun, Jianzhu Yao, Franck Dernoncourt, Satyapriya Krishna, Sina Rismanchian, Bonan Pu, Francesco Pinto, Yingheng Wang, Kumar Shridhar, Kalon J. Overholt, Glib Briia, Hieu Nguyen, David, Soler Bartomeu, Tony CY Pang, Adam Wecker, Yifan Xiong, Fanfei Li, Lukas S. Huber, Joshua Jaeger, Romano De Maddalena, Xing Han Lù, Yuhui Zhang, Claas Beger, Patrick Tser Jern Kon, Sean Li, Vivek Sanker, Ming Yin, Yihao Liang, Xinlu Zhang, Ankit Agrawal, Li S. Yifei, Zechen Zhang, Mu Cai, Yasin Sonmez, Costin Cozianu, Changhao Li, Alex Slen, Shoubin Yu, Hyun Kyu Park, Gabriele Sarti, Marcin Briański, Alessandro Stolfo, Truong An Nguyen, Mike Zhang, Yotam Perlitz, Jose Hernandez-Orallo, Runjia Li, Amin Shabani, Felix Juefei-Xu, Shikhar Dhingra, Orr Zohar, My Chiffon Nguyen, Alexander Pondaven, Abdurrahim Yilmaz, Xuandong Zhao, Chuanyang Jin, Muyan Jiang, Stefan Todoran, Xinyao Han, Jules Kreuer, Brian Rabern, Anna Plassart, Martino Maggetti, Luther Yap, Robert Geirhos, Jonathon Kean, Dingsu Wang, Sina Mollaei, Chenkai Sun, Yifan Yin, Shiqi Wang, Rui Li, Yaowen Chang, Anjiang Wei, Alice Bizeul, Xiaohan Wang, Alexandre Oliveira Arrais, Kushin Mukherjee, Jorge Chamorro-Padial, Jiachen Liu, Xingyu Qu, Junyi Guan, Adam Bouyamourn, Shuyu Wu, Martyna Plomecka, Junda Chen, Mengze Tang, Jiaqi Deng, Shreyas Subramanian, Haocheng Xi, Haoxuan Chen, Weizhi Zhang, Yinuo Ren, Haoqin Tu, Sejong Kim, Yushun Chen, Sara Vera Marjanović, Junwoo Ha, Grzegorz Luczyna, Jeff J. Ma, Zewen Shen, Dawn Song, Cedegao E. Zhang, Zhun Wang, Gaël Gendron, Yunze Xiao, Leo Smucker, Erica Weng, Kwok Hao Lee, Zhe Ye, Stefano Ermon, Ignacio D. Lopez-Miguel, Theo Knights, Anthony Gitter, Namkyu Park, Boyi Wei, Hongzheng Chen, Kunal Pai, Ahmed Elkhanany, Han Lin, Philipp D. Siedler, Jichao Fang, Ritwik Mishra, Károly Zsolnai-Fehér, Xilin Jiang, Shadab Khan, Jun Yuan, Rishab Kumar Jain, Xi Lin, Mike Peterson, Zhe Wang, Aditya Malusare, Maosen Tang, Isha Gupta, Ivan Fosin, Timothy Kang, Barbara Dworakowska, Kazuki Matsumoto, Guangyao Zheng, Gerben Sewuster, Jorge Pretel Villanueva, Ivan Rannev, Igor Chernyavsky, Jiale Chen, Deepayan Banik, Ben Racz, Wenchao Dong, Jianxin Wang, Laila Bashmal, Duarte V. Gonçalves, Wei Hu, Kaushik Bar, Ondrej Bohdal, Atharv Singh Patlan, Shehzaad Dhuliawala, Caroline Geirhos, Julien Wist, Yuval Kansal, Bingsen Chen, Kutay Tire, Atak Talay Yücel, Brandon Christof, Veerupaksh Singla, Zijian Song, Sanxing Chen, Jiaxin Ge, Kaustubh Ponkshe, Isaac Park, Tianneng Shi, Martin Q. Ma, Joshua Mak, Sherwin Lai, Antoine Moulin, Zhuo Cheng, Zhanda Zhu, Ziyi Zhang, Vaidehi Patil, Ketan Jha, Qiutong Men, Jiaxuan Wu, Tianchi Zhang, Bruno Hebling Vieira, Alham Fikri Aji, Jae-Won Chung, Mo-

- ammed Mahfoud, Ha Thi Hoang, Marc Sperzel, Wei Hao, Kristof Meding, Sihan Xu, Vassilis Kostakos, Davide Manini, Yueying Liu, Christopher Toukmaji, Jay Paek, Eunmi Yu, Arif Engin Demircali, Zhiyi Sun, Ivan Dewerpe, Hongsen Qin, Roman Pflugfelder, James Bailey, Johnathan Morris, Ville Heilala, Sybille Rosset, Zishun Yu, Peter E. Chen, Woongyeong Yeo, Eeshaan Jain, Ryan Yang, Sreekar Chigurupati, Julia Chernyavsky, Sai Prajwal Reddy, Subhashini Venugopalan, Hunar Batra, Core Francisco Park, Hieu Tran, Guilherme Maximiano, Genghan Zhang, Yizhuo Liang, Hu Shiyu, Rongwu Xu, Rui Pan, Siddharth Suresh, Ziqi Liu, Samaksh Gulati, Songyang Zhang, Peter Turchin, Christopher W. Bartlett, Christopher R. Scotese, Phuong M. Cao, Ben Wu, Jacek Karwowski, Davide Scaramuzza, Aakaash Nattanmai, Gordon McKellips, Anish Cheraku, Asim Suhail, Ethan Luo, Marvin Deng, Jason Luo, Ashley Zhang, Kavin Jindel, Jay Paek, Kasper Halevy, Allen Baranov, Michael Liu, Advait Avadhanam, David Zhang, Vincent Cheng, Brad Ma, Evan Fu, Liam Do, Joshua Lass, Hubert Yang, Surya Sunkari, Vishruth Bharath, Violet Ai, James Leung, Rishit Agrawal, Alan Zhou, Kevin Chen, Tejas Kalpathi, Ziqi Xu, Gavin Wang, Tyler Xiao, Erik Maung, Sam Lee, Ryan Yang, Roy Yue, Ben Zhao, Julia Yoon, Sunny Sun, Aryan Singh, Ethan Luo, Clark Peng, Tyler Osbey, Taozhi Wang, Daryl Echeazu, Hubert Yang, Timothy Wu, Spandan Patel, Vidhi Kulkarni, Vijaykaarti Sundarapandian, Ashley Zhang, Andrew Le, Zafir Nasim, Srikar Yalam, Ritesh Kasamsetty, Soham Samal, Hubert Yang, David Sun, Nihar Shah, Abhijeet Saha, Alex Zhang, Leon Nguyen, Laasya Nagumalli, Kaixin Wang, Alan Zhou, Aidan Wu, Jason Luo, Anwith Telluri, Summer Yue, Alexandr Wang, and Dan Hendrycks. Humanity’s last exam, 2025. URL <https://arxiv.org/abs/2501.14249>.
- [17] Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. RL’s razor: Why online reinforcement learning forgets less, 2025. URL <https://arxiv.org/abs/2509.04259>.
- [18] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys ’25, pp. 1279–1297. ACM, March 2025. doi: 10.1145/3689031.3696075. URL <http://dx.doi.org/10.1145/3689031.3696075>.
- [19] Hongjin Su, Shizhe Diao, Ximing Lu, Mingjie Liu, Jiacheng Xu, Xin Dong, Yonggan Fu, Peter Belcak, Hanrong Ye, Hongxu Yin, Yi Dong, Evelina Bakhturina, Tao Yu, Yejin Choi, Jan Kautz, and Pavlo Molchanov. Toolorchestra: Elevating intelligence via efficient model and tool orchestration, 2025. URL <https://arxiv.org/abs/2511.21689>.
- [20] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. Shadowkv: Kv cache in shadows for high-throughput long-context llm inference, 2025. URL <https://arxiv.org/abs/2410.21465>.
- [21] 5 Team, Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, Kedong Wang, Lucen Zhong, Mingdao Liu, Rui Lu, Shulin Cao, Xiaohan Zhang, Xuancheng Huang, Yao Wei, Yean Cheng, Yifan An, Yilin Niu, Yuanhao Wen, Yushi Bai, Zhengxiao Du, Zihan Wang, Zilin Zhu, Bohan Zhang, Bosi Wen, Bowen Wu, Bowen Xu, Can Huang, Casey Zhao, Changpeng Cai, Chao Yu, Chen Li, Chendi Ge, Chenghua Huang, Chenhui Zhang, Chenxi Xu, Chenzheng Zhu, Chuang Li, Congfeng Yin, Daoyan Lin, Dayong Yang, Dazhi Jiang, Ding Ai, Erle Zhu, Fei Wang, Gengzheng Pan, Guo Wang, Hailong Sun, Haitao Li, Haiyang Li, Haiyi Hu, Hanyu Zhang, Hao Peng, Hao Tai, Haoke Zhang, Haoran Wang, Haoyu Yang, He Liu, He Zhao, Hongwei Liu, Hongxi Yan, Huan Liu, Huilong Chen, Ji Li, Jiajing Zhao, Jiamin Ren, Jian Jiao, Jiani Zhao, Jianyang Yan, Jiaqi Wang, Jiayi Gui, Jiayue Zhao, Jie Liu, Jijie Li, Jing Li, Jing Lu, Jingsen Wang, Jingwei Yuan, Jingxuan Li, Jingzhao Du, Jinhua Du, Jinxin Liu, Junkai Zhi, Junli Gao, Ke Wang, Lekang Yang, Liang Xu, Lin Fan, Lindong Wu, Lintao Ding, Lu Wang, Man Zhang, Minghao Li, Minghuan Xu, Mingming Zhao, Mingshu Zhai, Pengfan Du, Qian Dong, Shangde Lei, Shangqing Tu, Shangtong Yang, Shaoyou Lu, Shijie Li, Shuang Li, Shuang-Li, Shuxun Yang, Siboyi, Tianshu Yu, Wei Tian, Weihang Wang, Wenbo Yu, Weng Lam Tam, Wenjie Liang, Wentao Liu, Xiao Wang, Xiaohan Jia, Xiaotao Gu, Xiaoying Ling, Xin Wang, Xing Fan, Xingru Pan, Xinyuan Zhang, Xinze Zhang, Xiuqing Fu, Xunkai Zhang, Yabo Xu, Yandong Wu, Yida Lu, Yidong Wang, Yilin Zhou, Yiming Pan, Ying Zhang, Yingli Wang, Yingru Li, Yinpei Su, Yipeng Geng, Yitong Zhu, Yongkun Yang, Yuhang Li, Yuhao Wu, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yuxuan Zhang, Zezhen Liu, Zhen Yang, Zhengda Zhou, Zhongpei Qiao,

- Zhuoer Feng, Zhuorui Liu, Zichen Zhang, Zihan Wang, Zijun Yao, Zikang Wang, Ziqiang Liu, Ziwei Chai, Zixuan Li, Zuodong Zhao, Wenguang Chen, Jidong Zhai, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, Yuxiao Dong, and Jie Tang. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models, 2025. URL <https://arxiv.org/abs/2508.06471>.
- [22] vLLM Team. KV-aware Routing — vLLM Production Stack Documentation. <https://docs.vllm.ai/projects/production-stack/en/vllm-stack-0.1.5/tutorials/kvaware.html>, 2025. Accessed: 2026-01-15.
- [23] Weixun Wang, XiaoXiao Xu, Wanhe An, Fangwen Dai, Wei Gao, Yancheng He, Ju Huang, Qiang Ji, Hanqi Jin, Xiaoyang Li, et al. Let it flow: Agentic crafting on rock and roll, building the rome model within an open agentic learning ecosystem. *arXiv preprint arXiv:2512.24873*, 2025.
- [24] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents, 2025. URL <https://arxiv.org/abs/2407.16741>.
- [25] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Oworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL <https://arxiv.org/abs/2404.07972>.
- [26] Zhiqiang Xie, Ziyi Xu, Mark Zhao, Yuwei An, Vikram Sharma Mailthody, Scott Mahlke, Michael Garland, and Christos Kozyrakis. Strata: Hierarchical context caching for long context language model serving, 2025. URL <https://arxiv.org/abs/2508.18572>.
- [27] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- [28] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- [29] Lingfan Yu, Jinkun Lin, and Jinyang Li. Stateful large language model serving with pensieve. In *Proceedings of the Twentieth European Conference on Computer Systems, EuroSys ’25*, pp. 144–158. ACM, March 2025. doi: 10.1145/3689031.3696086. URL <http://dx.doi.org/10.1145/3689031.3696086>.
- [30] Yichao Yuan, Advait Iyer, Lin Ma, and Nishil Talati. Vortex: Overcoming memory capacity limitations in gpu-accelerated large-scale data analytics, 2025. URL <https://arxiv.org/abs/2502.09541>.
- [31] Hanchen Zhang, Xiao Liu, Bowen Lv, Xueqiao Sun, Bohao Jing, Iat Long Iong, Zhenyu Hou, Zehan Qi, Hanyu Lai, Yifan Xu, et al. Agentrl: Scaling agentic reinforcement learning with a multi-turn, multi-task framework. *arXiv preprint arXiv:2510.04206*, 2025.
- [32] Lei Zhang, Mouxiang Chen, Ruisheng Cao, Jiawei Chen, Fan Zhou, Yiheng Xu, Jiaxi Yang, Liang Chen, Changwei Luo, Kai Zhang, et al. Megaflow: Large-scale distributed orchestration system for the agentic era. *arXiv preprint arXiv:2601.07526*, 2026.

- [33] Chujie Zheng, Kai Dang, Bowen Yu, Mingze Li, Huiqiang Jiang, Junrong Lin, Yuqiong Liu, Hao Lin, Chencan Wu, Feng Hu, An Yang, Jingren Zhou, and Junyang Lin. Stabilizing reinforcement learning with llms: Formulation and practices, 2025. URL <https://arxiv.org/abs/2512.01374>.
- [34] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs, 2024. URL <https://arxiv.org/abs/2312.07104>.
- [35] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving, 2024. URL <https://arxiv.org/abs/2401.09670>.
- [36] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents, 2024. URL <https://arxiv.org/abs/2307.13854>.

A 扩展讨论

A.1 KV 缓存优化

多层 KV 缓存管理。 为缓解 GPU 内存压力, Pensieve [29]、Continuum [12]、Strata [26] 和 ShadowKV [20] 等系统利用 GPU HBM、CPU DRAM 及 NVMe SSD 组成的硬件内存层次管理 KV 缓存, 将非活跃 KV 状态卸载到低层存储, 并在请求恢复时预取回 GPU, 以缓解暂时性抢占。然而, 实际效率受到设备与主机之间层间带宽的根本限制。高频代理工作流中, 频繁换入换出的开销常抵消多层缓存收益, 见 节 5。

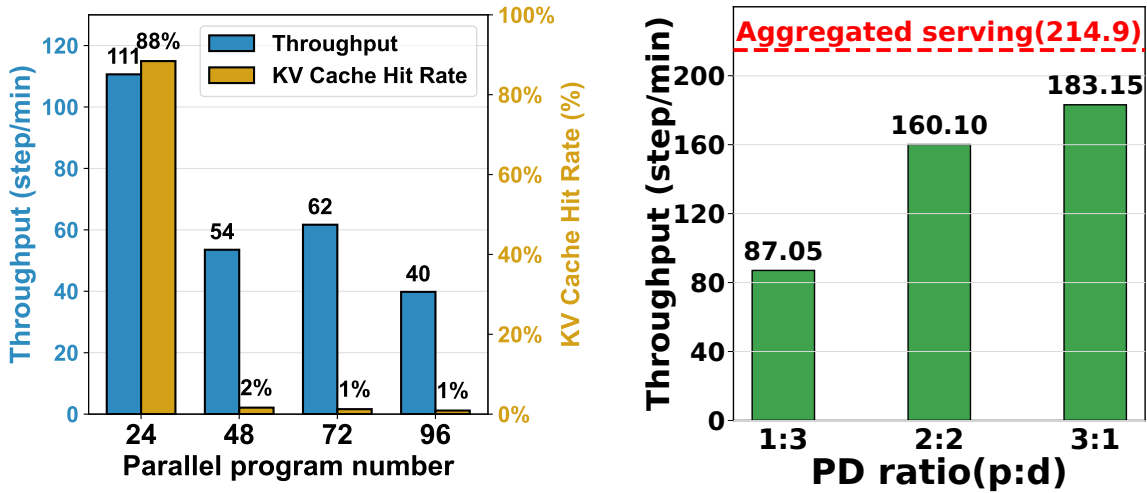
分布式 KV 缓存管理。 分布式代理状态管理使 KV 缓存逐出及抢占策略明显复杂化。BanaServe [7]、LMCache [13] 等支持 DP 节点间缓存传输, 但大批量代理服务及 rollout 的性能常受互连带宽限制。代理程序内部依赖很强, 缺乏程序级管理时需频繁转移状态, 容易使服务或 rollout 期间的网络饱和。

为避开带宽瓶颈, vLLM KV 感知路由器 [22]、SGLang Model Gateway [34] 等标准推理系统, 按前缀局部性或会话 ID 将请求固定到特定节点。Vortex [30] 则通过会话感知预取减少跨节点传输延迟。但它们无法在 DP 节点间动态迁移活跃程序状态, 因而产生图 图 2a 所示的严重内存利用率不均衡。承载长期代理程序的节点无法将状态移到闲置节点, 导致资源利用碎片化和总吞吐量下降。

A.2 现有 KV 缓存优化方法

KV 缓存卸载。 我们用 LMcache [13] 调研卸载能否缓解容量约束。利用 CPU 或 SSD 理论上可扩展有效内存, 但 vLLM + LMcache 实现暴露出关键瓶颈: PCIe 带宽不足以支撑代理负载中高频上下文切换和大量数据传输。如 图 7a 所示, 用 GLM-4.6 [21] 和 mini-SWEAgent [28] 提供服务时, 频繁换入换出的延迟抵消容量收益, 重代理负载下吞吐量严重下降。

预填充与解码分离。 还研究了预填充与解码 (PD) 分离 [35], 这一常见聊天服务优化通过隔离预填充干扰来改善解码。但对于上下文持续增长的代理负载, 我们观察到它加剧抖动。集群分为纯预填充和纯解码节点后, 可处理预填充的有效 HBM 池比统一架构小得多; 这种内存分割使系统在更低并发度下触及容量上限并发生抖动, 见 图 7b。结果说明, 通用架构优化不能取代主动管理工作集的程序中心调度器。



(a) 启用 KV 缓存卸载时的命中率

(b) 吞吐量与纯预填充/纯解码节点比例的关系

图 7: KV 缓存卸载及预填充与解码 (PD) 分离的消融研究。

A.3 代理工作流的规模扩展

异构资源分配与调度。 为大规模编排多轮代理环境交互, MegaFlow [32]、RollArt [6, 23]、AgentRL [31] 和 VerlTool [9] 等近期系统将模型推理与环境执行解耦。虽然专门服务能够有效扩展环境并发, 但框架仍受粗

粒度分离的固有限制：把推理引擎和工具执行器视为独立黑盒，缺乏统一资源管理，不能协调 KV 缓存生命周期与环境执行。没有细粒度程序级调度，基于分离的方法会浪费代理负载的 KV 复用潜力，吞吐量不理想。

ThunderAgent 作为路由策略的可扩展性。 多节点部署时，ThunderAgent 用全局等待队列取代基于会话的静态节点固定。暂停工作流准备恢复时，将请求路由到可用容量最多的节点。这一策略在 KV 局部性与多节点负载均衡之间折中，兼顾有效缓存利用和均匀负载分布。

正文分布式 rollout 结果 (表 2) 采用标准 2 节点设置；这里用相同 GLM-4.6 + mini-SWEAgent 负载，扩展至最多 8 节点 (64×H100)，按节点数同比扩展并发，每节点 72 个程序。基线为 SGLang Model Gateway 的缓存感知路由策略²，它按前缀局部性将请求固定于特定副本，无法跨节点重新分配暂停程序。

图 8 将吞吐量与基线及理想线性扩展参照比较。从 2 到 8 节点，THUNDERAGENT 接近线性扩展线；缓存感知路由策略则迅速落后，因为内存不均随集群规模增长，仅考虑局部性的路由器无法均衡暂停程序池，参见节 G.1。具体而言，相对 SGLang 的加速由 2 节点时的 1.79× 扩大至 8 节点时的 2.39×。还测量了全局调度器每次检测的通信延迟：从 2 节点的 13.5 ms 亚线性增长至 8 节点的 22.7 ms；节点增为 4×，延迟仅增为 1.68×，仍远低于秒级单步推理延迟。因此，全局调度器不是扩展瓶颈。

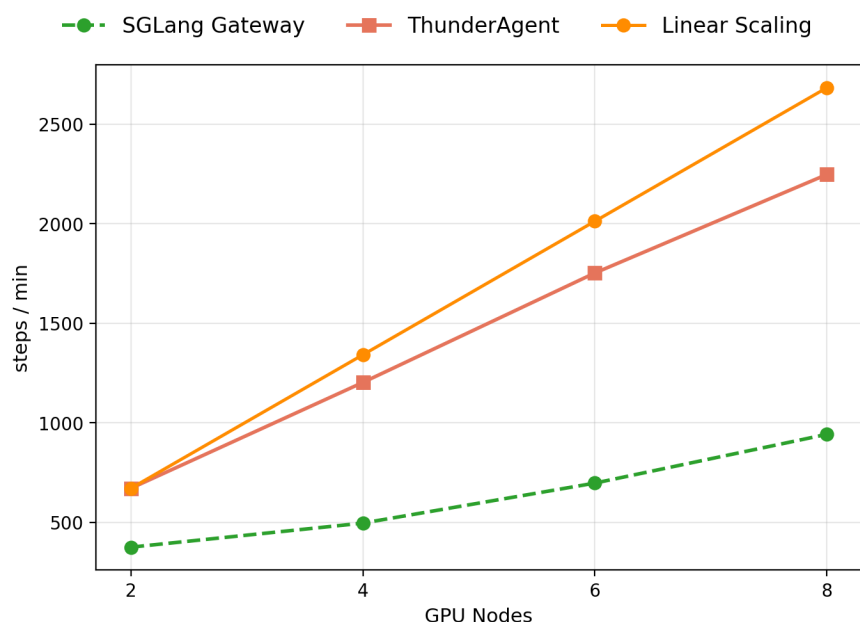


图 8: 将 ThunderAgent 扩展至 8 个 H100 节点，负载为 GLM-4.6 + mini-SWEAgent，每节点 72 个并发程序。THUNDERAGENT 接近理想线性参照，而 SGLang Model Gateway 缓存感知路由随集群增大逐渐落后。

A.4 ThunderAgent 与 KV 缓存卸载的兼容性

THUNDERAGENT 与 KV 缓存卸载相互独立，启用卸载后仍有效。为展示这一点，将 THUNDERAGENT 集成到 SGLang HiCache [26]，主机内存比例设为 2.0，在 8×H100 上使用 MiniMax M2.5 服务 R2E-Gym。服务批大小从 64 扫至 192，与相同 SGLang + HiCache 软件栈未启用 THUNDERAGENT 的基线比较。

高并发时，仅卸载仍不够。 图 9 显示，批大小不超过 96 时，主机和设备的联合容量仍能容纳工作集，基线维持高吞吐量。但批大小为 160 和 192 时联合容量耗尽：P50 延迟从 ~10 s 跃至批 160 的 ~30 s 和批 192 的 ~70 s，尽管批更大，吞吐量反而下降。

ThunderAgent 避免容量崩溃，并在每个批大小下改善吞吐量。 通过暂时暂停短程序，使活跃工作集保持在可用容量内，THUNDERAGENT 在整个范围内将 P50、P90 延迟分别稳定维持于 ~10 s 和 ~20 s，吞吐量随批大小单调增加。即使小批时两者通过 PCIe 交换都获得高命中率，选择性暂停行动程序仍能改善吞吐量和延迟。

²https://docs.sglang.io/docs/advanced_features/sgl_model_gateway#cache-aware-policy-tuning

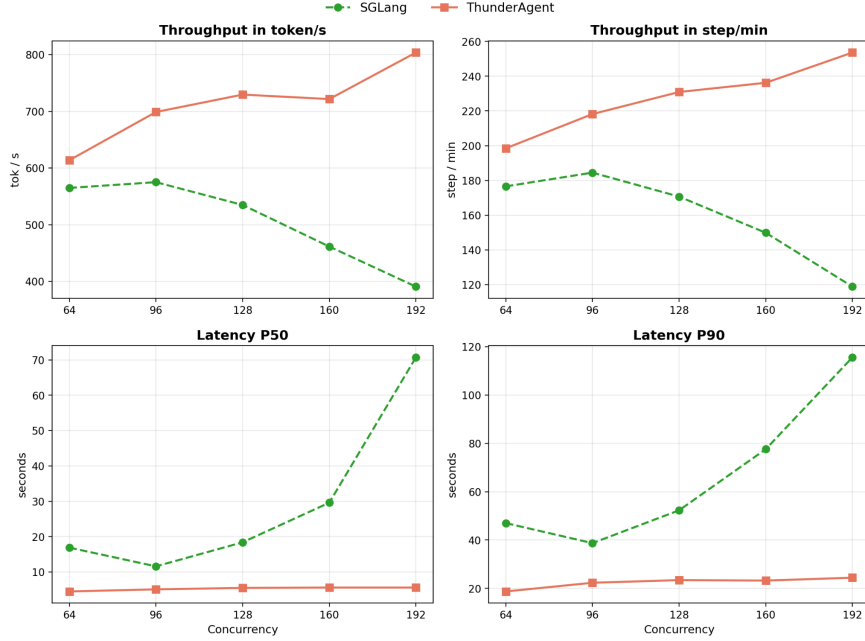


图 9: THUNDERAGENT 与 SGLang HiCache 结合。

A.5 ThunderAgent 跨硬件代际的可移植性

还在计算能力与带宽比低于 H100 的硬件上测试 THUNDERAGENT。表 3 报告两类 GPU 的相关比值：A100 为 153 GFLOPS/GB，约为 H100 的 295 的一半。

图 10 给出 8×A100 上与图 4 的 a、b 相同负载的吞吐量。并发 24 时，工作集轻松容纳于 HBM，抖动很少，两条流水线上 THUNDERAGENT 与 vLLM 的差异均在噪声范围内。并发增为 48、72 时，THUNDERAGENT 的优势扩大：mini-SWEAgent 为 1.71–2.08×，OpenHands 为 1.16–1.62×；而 vLLM 从并发 48 起开始抖动，吞吐量下降。

GPU	计算 (TFLOPs)	带宽 (GB/s)	比值
H100	989	3350	295.2
A100	312	2039	153.0

表 3: 不同硬件的计算与带宽比。计算能力采用无稀疏性的 FP16 Tensor Core 性能，单位为 TFLOPs；带宽为全局内存到 L2 的带宽，单位为 GB/s。

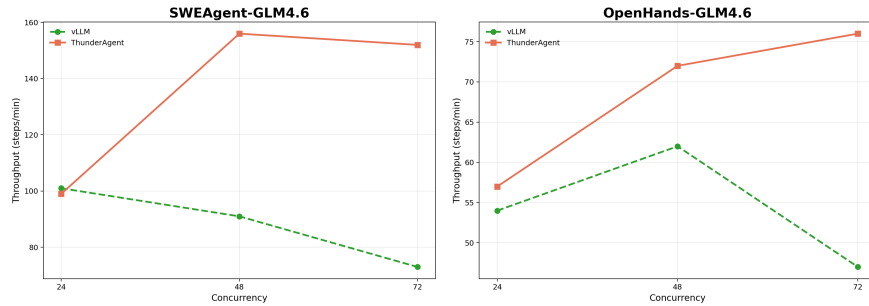


图 10: A100 GPU 上的 THUNDERAGENT。

A.6 开源框架对 ThunderAgent 的采用

除本文评估外，开源强化学习及服务框架已采用 THUNDERAGENT 加速代理 rollout 和服务，说明仅需较小接口修改即可将其集成到既有软件栈。

SkyRL(强化学习 rollout)。 ThunderAgent 已集成到 SkyRL [3] 主仓库的 `examples/train/thunder_agent` 下，示例训练方案见：https://github.com/NovaSky-AI/SkyRL/tree/main/examples/train/thunder_agent。该方案将 THUNDERAGENT 用作 SkyRL 训练的 rollout 网关；通过 `program_id` 将每条代理轨迹暴露为程序，终止后显式释放，对应 附录 B 的最小接口。已发布方案用 SkyRL 完全异步框架，在 R2EGym 上训练 Qwen3-32B，相对基线获得 $3.01\times$ 的实际运行时间加速，说明只需极少接口修改即可采用 THUNDERAGENT。

NVIDIA Dynamo (服务)。 作为 Dynamo 2.0 的一部分³，THUNDERAGENT 正在集成到数据中心规模的分布式推理框架 NVIDIA Dynamo⁴，以提高代理负载的推理吞吐量。THUNDERAGENT 提出的程序抽象被标准化为服务协议 `nvext.agent_context` 的一部分⁵，作为 Dynamo 的一等调度单元。

³<https://github.com/ai-dynamo/dynamo/issues/9208>

⁴<https://github.com/ai-dynamo/dynamo/pull/9448>

⁵<https://github.com/ai-dynamo/dynamo/pull/8789>

B 系统可移植性与接口抽象

B.1 中间件架构与统一接口

THUNDERAGENT 是**程序感知运行时层**，通过程序级抽象，在代理控制流和后端推理引擎之间协调。调度器结合抽象的 ProgramState（表 4a）和后端缓存容量视图（表 5），控制程序状态转移。每个程序只绑定端点，不依赖具体后端实现。

B.2 程序 ID 为什么重要

标准会话 ID 主要作为路由标签，而**程序 ID 供本文系统检查 workflow 元数据**。这种可见性使调度器能够区分有效工具等待与闲置会话，实现基于会话的基线无法支持的智能抢占。

仅有推理后端（例如 vLLM/SGLang）	加入 ThunderAgent
<pre> 1 # 1) LLM 请求 2 chat_completion(model_id, messages, extrabody) 3 4 # 2) 工具执行 5 run_tool(command, sandbox) 6 7 # 3) 程序结束 #（不显式释放） </pre>	<pre> 1 # 1) LLM 请求 2 extrabody["program_id"] = "PID" 3 chat_completion(model_id, messages, extrabody) 4 5 # 2) 工具执行 6 run_tool(command, sandbox, program_id="PID") 7 8 # 3) 程序结束（显式释放） 9 POST /programs/release 10 { "program_id": "PID" } </pre>

图 11: 使用 ThunderAgent 只需三处修改。

B.3 低开销采用 ThunderAgent

图 11 显示，采用 ThunderAgent 只需为 LLM 推理及工具执行请求附加 program_id，并在程序结束时发送带 program_id 的显式释放信号。该标识将每个请求归属到程序实例，供调度使用；释放信号让 ThunderAgent 终止后回收每程序资源。**其他请求字段及 OpenAI 风格 API 接口均无需改变。**

字段	类型	含义	状态	含义
ProgramState			ProgramStatus	
status	ProgramStatus	当前生命周期状态。	REASONING	GPU 上推理。
backend_url	str	分配的后端端点。	ACTING	GPU 外执行工具。
step_count	int	已执行步骤数。	PAUSED	位于全局暂停等待集合。
total_tokens	int	完整历史的总词元数。	STOPPED	已释放，资源已回收。

(a) ProgramState 字段。

(b) ProgramStatus 的语义。

表 4: 程序状态及状态类型定义。

字段	类型	含义
BackendState		
url	str	后端端点。
healthy	bool	用于调度的健康标志。
cache_config	Optional[CacheConfig]	启动时获取的静态缓存配置。
active_program_tokens	int	后端上活跃词元的占用。

表 5: BackendState 的关键字段。

C 工具执行时间的变化

真实代理工具调用难以刻画,且经常不可预测。某些编程场景,例如在 SWE-Bench [10] 上使用 SWE-agent [28] 或 OpenHands [24], 主要调用本地轻量工具, 延迟较稳定、方差较低。但在更广泛、真实的场景中, 例如在 HLE [16] 上使用 ToolOrchestra [19], 工作负载更依赖远程服务工具 (表 6), 工具时长波动大、难预测。波动主要来自代理运行时之外的网络抖动、后端负载、排队和速率限制, 它们随请求及时间变化。

图 12 从实证上验证这一现象。远程服务工具及部分执行工具中, 中位数与尾部分位数相差很大; p95、p99 明显高于中位数, 尾部可延伸至数十乃至数百秒。这表明此类工具延迟缺乏稳定的中心趋势, 由重尾行为主导, **使实践中的工具延迟预测本身十分脆弱**。

工具执行不可预测时, 低估时长会浪费已经固定的缓存容量, 仍使 KV 缓存过早逐出, 恢复时引发抖动; 相反, 高估时长可能造成本应继续固定的程序 KV 缓存不必要地被逐出。即使工具时长完全可预测, continuum [12] 等方法仍用静态阈值规则决定是否继续固定。ThunderAgent 则构建**完整成本建模框架**, **动态权衡** $\text{Cost}_{\text{recompute}}$ 与 $\text{Cost}_{\text{caching}}$ 。

工具类别	作用	主要变化来源
HLE-search	检索证据	远程服务 (网络延迟/限流)
HLE-enhance-reasoning	将模型作为工具调用	远程服务
HLE-answer	最终生成	本地 LLM 推理
SAB-execute_bash	Shell 执行	沙箱与 I/O
SAB-execute_ipython_cell	Python 单元执行	程序运行时间
SAB-str_replace_editor	文件编辑	本地文件系统
SAB-task_tracker	任务状态跟踪	本地文件系统

表 6: 工具类别。

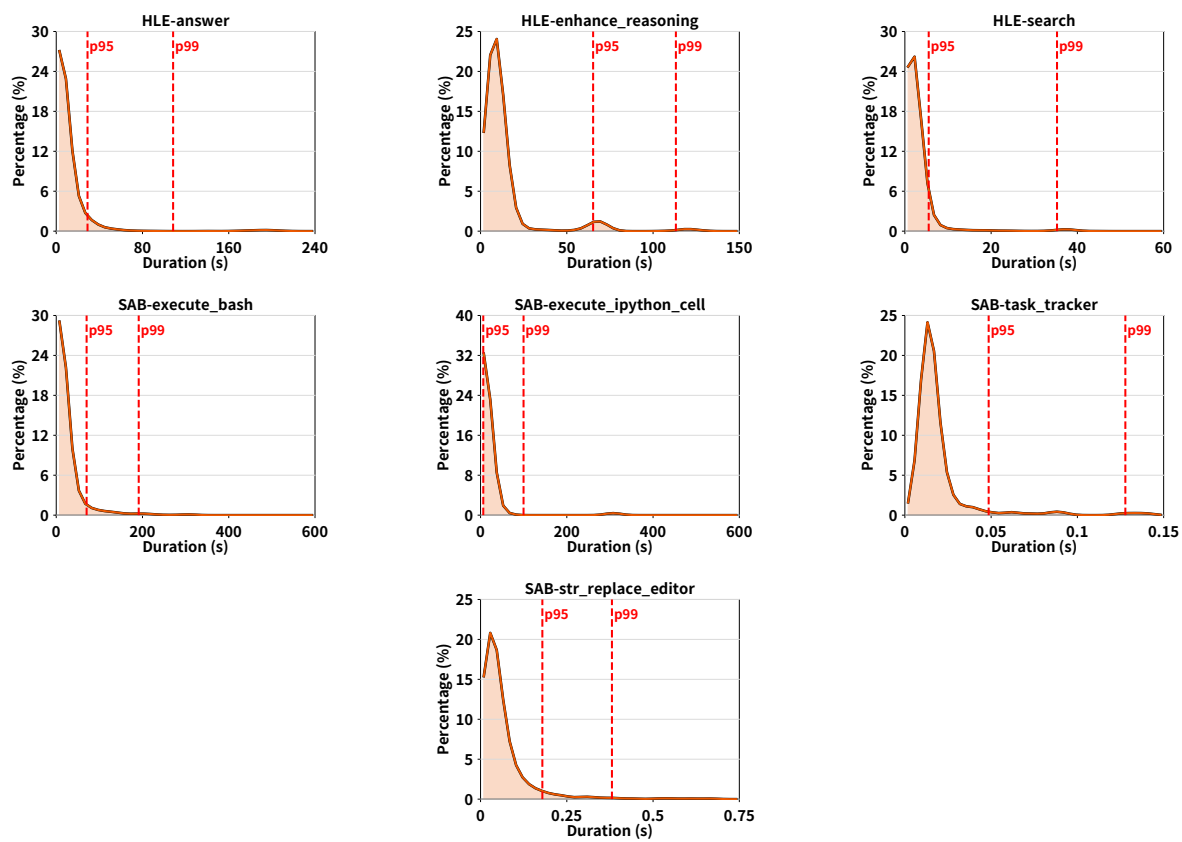


图 12: 工具执行时间分布。工具执行时间变化显著，难以预测。

D KV 缓存命中率的统计与解释

在成本分解式 (3) 中, 代理服务吞吐量损失主要来自无产出开销: 缓存抖动引起的 KV 重计算, 及外部工具执行期间的闲置 KV 保留, 即 $\text{Cost}_{\text{recompute}}$ 和 $\text{Cost}_{\text{caching}}$ 。工具调用短且可预测时, 行动阶段占用 KV 的时间短, $\text{Cost}_{\text{caching}}$ 小, 因此主要应避免抖动; 较高命中率通常意味着较少再次预填充及更高吞吐量。

但工具时长高度变化时, 见附录 C, 基于 TTL 的调度器可能为长工具调用持续固定 KV 缓存。它虽然减少 $\text{Cost}_{\text{recompute}}$ 、提高命中率, 却同时增大 $\text{Cost}_{\text{caching}}$ 、降低吞吐量。这解释了 continuum [12] 为什么在工具密集负载上, 即使命中率更高仍可能较慢, 见图 4、5。

ThunderAgent 显式平衡缓存与重计算, 适应这些不同情形。第 4.3 节为行动程序引入时间衰减 $f(t)$, 权衡 $\text{Cost}_{\text{caching}}$ 与 $\text{Cost}_{\text{recompute}}$; 附录 F.1 严格推导 $f(t)$ 的最优函数形式。通过逐渐降低长时间闲置行动程序的有效内存优先级, 调度器逐出其 KV 缓存, 在控制重计算的同时减少闲置缓存成本, 实际获得更好吞吐量, 见图 4。

E 端到端延迟分析

虽然 节 1 已指出, 对自主代理和代理强化学习 rollout, 程序级延迟, 即生成完整工作流所需时间, 远比每一步端到端延迟重要, 这里仍比较 THUNDERAGENT 与 vLLM、Continuum 的平均单步延迟。图 13 显示, 在一个 H100 上, 用 GLM4.6 和 Qwen3 235B 运行 mini-SWEAgent 与 Openhands, 不论并行工作流数高低, THUNDERAGENT 都明显优于基线。原因在于, 切换行动程序看似能改善端到端延迟, 实际上却会触发严重 KV 缓存抖动, 延迟所有运行中程序。

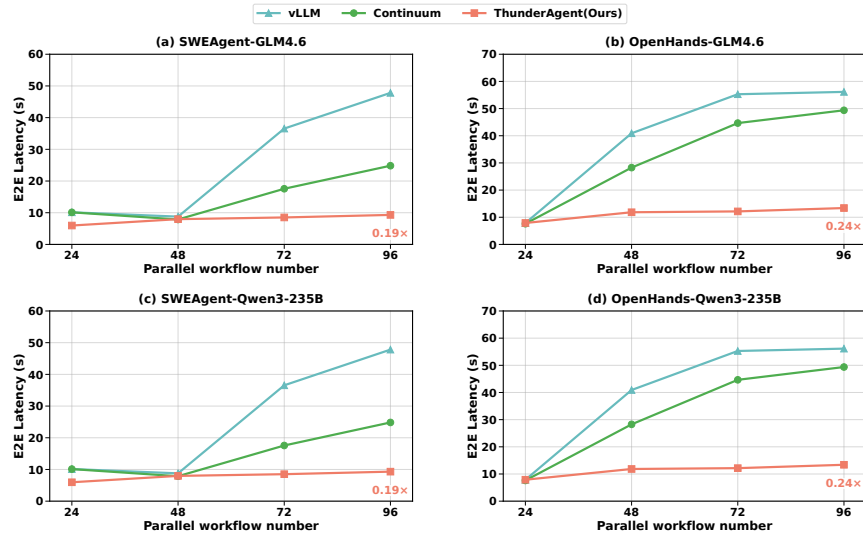


图 13: 端到端延迟比较。

F 扩展理论分析

F.1 周期性抖动检测中时间衰减函数的证明

假设 F.1 (不可预测的工具执行时间). 对于行动程序, 我们假设调度器无法可靠预测其工具返回时间, 见附录 C。因此, 衰减函数 f 应仅依赖已用行动时间 t , 且满足时间齐次性 [15]。

假设 F.2 (边界条件). 假设时间衰减函数 $f: [0, \infty) \rightarrow (0, 1]$ 满足:

$$f(0) = 1, \lim_{t \rightarrow \infty} f(t) = 0, \quad (12)$$

直观上, 工具执行时间为 0 时对应不调用工具的多轮交互, 所有行动程序都退化为推理程序, 因此 $f(t) = 1$ 。反之, 工具执行时间无穷大时, 请求永远不会返回下一轮, 工作流退化为类似标准聊天服务的单轮生成; 此时设置 $f(t) = 0$, 使衰减函数与请求级调度策略一致。

定理 F.1 (可接受的时间衰减函数). 在假设 F.1 与 F.2 下, 式 7 容量检查函数中可接受的衰减函数 f 必须为以下形式之一: 连续时间的指数衰减 $f(t) = e^{-\lambda t}$, $\lambda > 0$; 或离散检测时间的几何衰减 $f(k) = x^{-k}$, $x > 1$ 。

证明. 证明先形式化假设 F.1 所隐含的时间齐次性, 再根据假设 F.2 的边界条件推导允许的时间衰减函数 f 。
形式化不可预测的工具时间. 令 t 为已用行动时间, 以连续实际时间或离散周期检测次数度量。根据假设 F.1, 再等待 Δ 后的相对衰减不应依赖绝对已用时间 t , 只依赖增量 Δ 。将其形式化为存在函数 $\phi: [0, \infty) \rightarrow (0, 1]$, 使所有 $t, \Delta \geq 0$ 满足:

$$f(t + \Delta) = f(t) \phi(\Delta). \quad (13)$$

半群方程. 在式 13 中令 $t = 0$, 利用假设 F.2 中的边界条件 $f(0) = 1$, 得到 $\phi(\Delta) = f(\Delta)$ 。代回可得乘法半群方程:

$$f(t + \Delta) = f(t) f(\Delta), \quad \forall t, \Delta \geq 0. \quad (14)$$

连续时间情形: 指数衰减. 先考虑连续时间。定义 $h(t) \triangleq \ln f(t)$, 对式 14 两侧取对数, 得到柯西函数方程:

$$h(t + \Delta) = h(t) + h(\Delta). \quad (15)$$

由于 $f(t) \in (0, 1]$, 所有 $t \geq 0$ 均有 $h(t) \leq 0$, 所以 h 在 $[0, \infty)$ 上有上界。在这一有界性条件下, 柯西函数方程只有线性解 $h(t) = ct$, 其中 $c \in \mathbb{R}$ 。记 $\lambda \triangleq -c \geq 0$, 得到:

$$f(t) = e^{-\lambda t}. \quad (16)$$

最后, 边界条件 $\lim_{t \rightarrow \infty} f(t) = 0$ (假设 F.2) 排除 $\lambda = 0$, 因此 $\lambda > 0$ 。

离散时间情形: 几何衰减. 再考虑离散时间, 以整数检测次数 $k \in \mathbb{Z}_{\geq 0}$ 度量已用行动时间。式 14 变为:

$$f(m + n) = f(m) f(n), \quad \forall m, n \in \mathbb{Z}_{\geq 0}. \quad (17)$$

令 $n = 1$, 得到递推 $f(k) = f(k-1)f(1)$ 。令 $\gamma \triangleq f(1)$, 则 $f(k) = f(1)^k \triangleq \gamma^k$ 。边界条件 $\lim_{k \rightarrow \infty} f(k) = 0$ 表明 $0 < \gamma < 1$ 。等价地, 可参数化为:

$$f(k) = x^{-k}, \quad x \triangleq \gamma^{-1} > 1. \quad (18)$$

证明完毕。 $\square$

F.2 重计算 STP 成本的证明

按 节 4.2 的定义, 重计算 STP 成本为:

$$\text{Cost}_{\text{recompute}} = \int_0^{t_{\text{recompute}}} c_i(t) dt \quad (19)$$

其中, $c_i(t)$ 表示瞬时成本, 与解码步骤成正比, 即 $c_i(t) \propto t$ 。这是因为分块预填充每次迭代处理固定数量的 KV 对, 累计计算随时间线性增加。积分后得到 $\text{Cost}_{\text{recompute}} \propto t_{\text{recompute}}^2$ 。给定关系 $t_{\text{recompute}} = c_i \times T_{\text{decode}} / \text{chunk}$, 其中 T_{decode} 和分块大小均为常数, 可得:

$$\text{Cost}_{\text{recompute}} \propto c_i^2$$

F.3 最小化重计算 STP 成本的证明

我们用**交换论证**严格证明最短优先逐出策略的最优性。

问题定义。选择待逐出的暂停程序子集 S ，使回收总内存满足 $\sum_{i \in S} c_i \geq \Delta C$ ，同时最小化总重计算成本 $J(S) = \sum_{i \in S} c_i^2$ 。成本函数 $f(x) = x^2$ 严格凸且超可加，即正数 a, b 满足 $(a + b)^2 > a^2 + b^2$ 。

定理。严格选择上下文长度 c_i 最小的程序，是最小化 $J(S)$ 的最优策略。

证明。反证，假设最优集合 S^* 不是最短程序集合。那么存在“长”程序 $p_{long} \in S^*$ 与尚可选但未选的“短”程序 $p_{short} \notin S^*$ ，满足 $c_{short} < c_{long}$ 。

通过交换或分解 p_{long} 构造新集合 S' 。由于 $c_{long} > c_{short}$ ，可以在概念上将 p_{long} 分为长度 c_{short} 的一段及余量 $r = c_{long} - c_{short}$ 。

用 p_{short} 以及理论上的余量 r 替换 p_{long} 会改变成本。根据平方函数凸性，考虑：

$$c_{long}^2 = (c_{short} + r)^2 = c_{short}^2 + r^2 + 2c_{short}r \quad (20)$$

由于 $c_{short} > 0$ 且 $r > 0$ ，交叉项 $2c_{short}r > 0$ ，因此：

$$c_{short}^2 + r^2 < c_{long}^2 \quad (21)$$

不等式表明，将较大的逐出目标 c_{long} 分成较小的 $c_{short} + r$ ，会严格降低平方和。在调度器中，这意味着：若用一个大程序满足 ΔC 内存约束，可以用总容量相同的可用小程序或其组合替换，严格降低惩罚。

反复应用交换，用较小未选程序替换最大已选程序，会单调降低 $J(S)$ 。只有不存在此类交换，即 S 完全由可用的最短上下文程序组成时，才达到全局最小值。

结论。最短优先策略全局最优，因为注意力的超线性成本 $O(L^2)$ 对碎片化的惩罚小于聚合。

G 补充消融研究

通过补充消融单独分析 THUNDERAGENT 各项设计的贡献，补充 节 5.4 的参数敏感性研究。

G.1 全局与局部等待队列

比较 THUNDERAGENT 的**全局**等待队列与**局部**等待队列：前者允许暂停程序在原后端以外的 DP 节点恢复，后者始终恢复到原节点。局部版本中，内存饱和和节点上的暂停程序必须等待原节点出现容量；全局队列则将它们重新分配到利用不足的节点。

如 图 14 所示，从 2 到 4 个 H100 节点，全局队列稳定改善吞吐量。4 节点时，收益扩大到 1.28 $\times$ ，反映跨节点内存不均衡随集群规模增长（图 2a），而全局队列直接缓解这一问题。

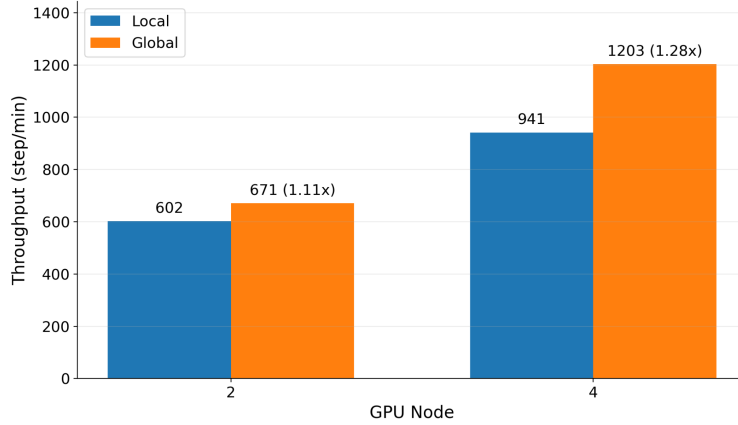


图 14: 全局与局部等待队列比较，采用 GLM-4.5-fp8 + mini-SWEAgent，每节点 8 张 H100、72 个并发程序。

G.2 时间衰减函数形式

定理 F.1 表明，在工具延迟不可预测的假设及假设 F.2 的边界条件下，指数衰减是 $f(t)$ 唯一允许的形式；这里将其与两个实际替代方案比较：**常数** ($f(t) = 1$ ，即不衰减)、**线性** ($f(t) = \max(1 - t, 0)$) 和**指数** ($f(t) = e^{-t}$)。采用工具延迟逐渐更难预测的三类流水线：mini-SWE-Agent，工具调用短而规律；OSWorld，调用较长但基本周期性，例如截图间隔固定 5s；ScienceAgent，调用高度随机、具有重尾，见 附录 C。

如 图 15 所示，指数衰减总体最稳健：mini-SWE-Agent 为 1.08 $\times$ ；ScienceAgent 为 1.63 $\times$ ，线性为 1.39 $\times$ 。不可预测或长尾时延揭示静态 $f(t)$ 的脆弱性。然而，在 OSWorld 上，指数比线性低约 6%，分别为 1.14 $\times$ 与 1.21 $\times$ ；工具时长接近确定时，线性递减权重更准确地跟踪可预测工具时钟。不衰减的常数形式从不回收长时间闲置行动程序的内存，在存在等待的各工作负载上均最弱。

G.3 逐出策略

从实证上将 节 F.3 中证明最优的最短优先逐出，与**随机**和**最长优先**两个自然基线比较。在一个 H100 节点上，用 GLM-4.5-fp8、72 个并发程序，运行 mini-SWE-Agent (*SWE*) 和 OpenHands。

按 $O(L^2)$ 重计算成本预测 (节 F.2)，逐出最长暂停程序最差：被丢弃 KV 状态的恢复重建最昂贵，增大平均预填充时间，并使解码流得不到资源。两条流水线上，最短优先的吞吐量最高、平均预填充时间最低 (图 16)。上下文最重的 OpenHands 差距最大：最短优先吞吐量约为最长优先的 2.0 $\times$ ，平均预填充时间缩短约 2 $\times$ ，从 15.1 s 至 7.5 s。随机策略介于二者之间，与它在相同成本曲面上充当无偏估计的性质一致。

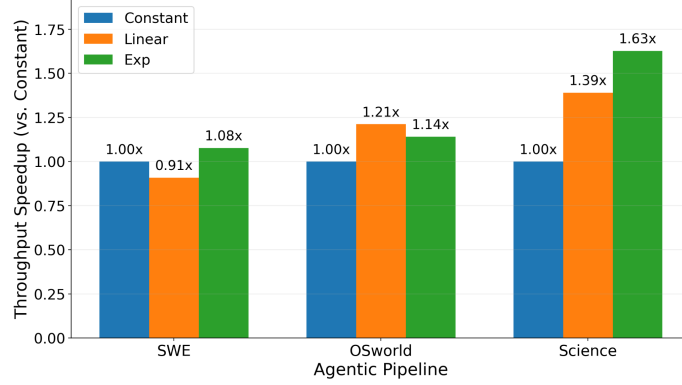


图 15: 三类代理流水线的衰减函数 $f(t)$ 消融。吞吐量报告为相对常数、不衰减基线的加速倍数。指数衰减总体最稳健；只有工具时间近乎确定的 OSWorld 更适合线性衰减。实验使用一个 8 张 H100 的节点、GLM-4.5-fp8 模型和 72 个并发程序。

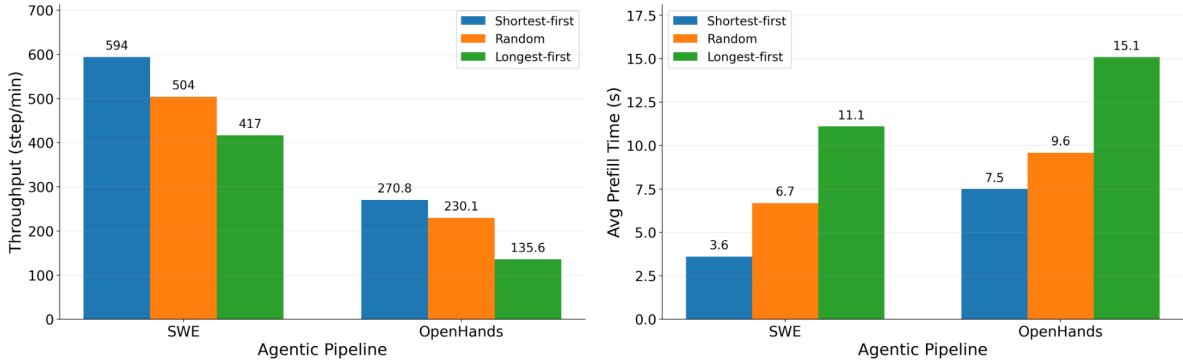


图 16: **逐出策略消融**。左为吞吐量，右为平均预填充时间。最短优先在两个指标上均最好，上下文更重的 OpenHands 差距更大。实验使用一个 8 张 H100 的节点、GLM-4.5-fp8 和 72 个并发程序。

G.4 各组件的贡献

从原生 vLLM 开始，依次启用**局部程序感知调度**，即每后端运行的成本驱动逐出与衰减函数，再启用**全局等待队列**，即跨节点程序迁移，以区分两项主要调度组件的贡献。表 7 报告 2×H100 节点、GLM-4.5-fp8 + mini-SWEAgent 和 144 个并发程序的吞吐量。

最大增幅来自局部程序感知调度：通过直接减少每个后端的不必要重计算和闲置缓存成本（式 3），吞吐量从 375 增至 602 步/分钟，即 1.61×。再加入全局队列，降低跨节点内存不均衡，又获得 1.12×，即 602 → 672；与 节 G.1 中独立比较全局与局部队列的结果一致。

组件	吞吐量（步/分钟）
vLLM（基线）	375
+ 局部程序感知调度	602
+ 全局等待队列（THUNDERAGENT）	672

表 7: 在 2×H100 + GLM-4.5-fp8 + mini-SWEAgent、144 个并发程序上的组件消融。加速相对 vLLM 报告。大部分收益来自局部调度；全局队列贡献其余跨节点不均衡改善。

图中英文与程序标识的中文对照

原始图像中的曲线、数值、标识及布局保留，以下给出正文和附录所有图内英文的中文含义。系统徽标无需翻译。

- 共同实验坐标与图例：Parallel workflow number / Parallel program number 为“并行工作流数/并行程序数”；Concurrency 为“并发数”；Throughput (step/min) / Throughput (steps/min) 为“吞吐量 (步/分钟)”；Throughput in token/s, tok/s 为“吞吐量 (词元/秒)”；Throughputs 为“吞吐量”；Speedup (x) 为“加速倍数”；E2E latency (s) 为“端到端延迟 (秒)”；Prefix cache hit rate (%), KV cache hit rate (%) 为“前缀缓存命中率/ KV 缓存命中率 (百分比)”；ThunderAgent (Ours)、Ours 为“ThunderAgent (本文)/本文”。SWEAgent、OpenHands、ToolOrchestra、ScienceAgent、模型、基线和数据集名称保留；code 为“编程”，HLE 是基准名。
- 内存与工具环境图：Time (min) 为“时间 (分钟)”；Memory usage (%) 为“内存占用 (百分比)”；Node0、Node1 为“节点 0、节点 1”；ABS Diff 为“绝对差值”；Processed workflows 为“已处理工作流”；Disk usage (TB) 为“磁盘占用 (TB)”；Active disk usage 为“活跃磁盘占用”；Disk memory capacity 为“磁盘容量”；Avg env prepare time(s) 为“平均环境准备时间 (秒)”。
- 系统概览：Global Waiting Queue 为“全局等待队列”；Backend 为“后端”；Inference Backend 为“推理后端”；Restore P6、P9 为“恢复程序 P6、P9”；Pause P2 为“暂停程序 P2”；Reasoning Program、Acting Program 为“推理程序、行动程序”；Pause、Restore 为“暂停、恢复”；Incoming Program 为“新到程序”；Thrashing 为“抖动”；Thrashing Monitor 为“抖动监控器”。HBM 为高带宽内存；示例中的 C、 τ 、 $\mathcal{L}$ 分别为上下文词元数、执行阶段及后端位置；15K、21K 为 15 千、21 千词元；Pydata_6、Astropy_1、Pydata_3、backend_3 及 P 编号是程序或后端标识。
- 延迟和参数消融图：Decode、Env、Prefill、Disk 为“解码、环境、预填充、磁盘”；Disk Usage 为“磁盘占用”； $x = 2$ and change Δt 为“固定 $x=2$ ，改变检测周期”； $\Delta t = 5$ and change x 为“固定检测周期为 5，改变 x ”； x in $f(t) = x^{-t}$ 为“衰减函数 $f(t)=x$ 的负 t 次幂中的 x ”。秒用 s 表示。
- 卸载及 PD 分离图：PD ratio(p:d) 为“预填充节点与解码节点比例”；Aggregated serving(214.9) 为“合并服务 (214.9)”；Throughput、KV Cache Hit Rate 分别为“吞吐量、KV 缓存命中率”。多节点图的 GPU Nodes 为“GPU 节点数”，Linear Scaling 为“线性扩展”。HiCache 图的 Latency P50、Latency P90 为“第 50、第 90 百分位延迟”；seconds 为“秒”。
- 工具分布图：Duration (s) 为“持续时间 (秒)”；Percentage (%) 为“百分比”；p95、p99 为“第 95、第 99 百分位”。HLE-answer 为最终回答，HLE-enhance_reasoning 为增强推理模型调用，HLE-search 为搜索；SAB-execute_bash、SAB-execute_ipython_cell、SAB-task_tracker、SAB-str_replace_editor 分别是 Shell 执行、Python 单元执行、任务跟踪和字符串替换编辑工具的标识。
- 补充消融图：Local、Global 为“局部、全局”；GPU Node 为“GPU 节点”；Constant、Linear、Exp 为“常数、线性、指数”；Throughput Speedup (vs. Constant) 为“相对常数策略的吞吐量加速”；Agentic Pipeline 为“代理流水线”；SWE、Osworld、Science 分别是软件工程、OSWorld、科学任务流水线；Shortest-first、Random、Longest-first 为“最短优先、随机、最长优先”；Avg Prefill Time (s) 为“平均预填充时间 (秒)”。
- 数学状态和接口：Active、Paused、Terminated 为“活跃、暂停、终止”；Reasoning、Acting 为“推理、行动”；ProgramState、ProgramStatus、BackendState 为“程序状态结构、程序状态类型、后端状态”；str、int、bool 为字符串、整数、布尔类型，Optional[CacheConfig] 为可选缓存配置。chat_completion、run_tool、POST /programs/release 分别表示聊天补全调用、工具运行调用和程序释放接口；字段及可执行接口名保留原样，含义已在附录表中翻译。

译者说明与原文论证的适用范围

本译本由当前对话模型逐段直接翻译，未调用本地语言模型或外部翻译服务。除正文外，扩展相关工作、可扩展性和硬件实验、接口示例、完整理论证明、补充消融、致谢与影响说明均纳入。数学、代码主体、引用及身份信息按原文保留。

关于最短优先的最优性。本文定义的是完整程序子集选择问题：释放总容量至少为 ΔC ，最小化被选程序上下文长度的平方和。附录的交换论证把长程序分成短程序和余量，但该余量未必是可选择真实程序，因此不能建立一般不可拆分子集问题的最优性。例如，可用程序长度只有 2 和 3，需释放 $\Delta C = 3$ ：按长度升序贪心先逐出 2 再逐出 3，总成本为 $2^2 + 3^2 = 13$ ；仅逐出长度 3 的程序已满足约束，成本为 $3^2 = 9$ 。因此，“严格选择最短程序全局最优”的表述在本文写出的约束下并不普遍成立。该策略的实验优势应与一般最优性分开评价，译文保留原主张和证明。

关于指数衰减。附录在“额外等待的相对衰减与已等待多久无关”的乘法时间齐次条件 $f(t + \Delta) = f(t)f(\Delta)$ 、正值有界性及给定边界条件下，推导指数/几何形式。这是带条件的函数形式结论；工具耗时难预测本身不直接推出该乘法公理，也没有证明指数形式对所有真实工具分布和吞吐量目标全局最优。原文的重计算二次成本推导还假设固定分块及固定每迭代时间，实际成本受硬件、模型和批处理影响。

关于状态、容量与迁移。全局队列是在缓存已被逐出的假设下将暂停程序恢复到其他节点，不等于无代价迁移活跃 KV 状态。时间衰减折减的是容量检查中的有效权重，实际缓存字节并未同步缩小。正文的两个水位在实现中都取 1，因而没有严格分离的滞回区间。“不发生抖动”的保证还需结合检测间隔、上下文增长及内存压力下的逐出条件理解。正文参数 $\Delta t = 5$ 与 $f(t) = 2^{-t}$ 需要结合实现的时间单位使用；附录同时讨论连续时间与离散检测次数，不能无条件互换。

关于比较与实验数字。正文和附录对 Continuum 的阈值与预测机制的描述是 ThunderAgent 作者的比较观点，应结合 Continuum 自身的成本收益和经验分布 TTL 模型阅读。附录称高估工具时间可能逐出本应保留的缓存，该方向性表述按原文保留。服务曲线中 ScienceAgent 的对 vLLM 倍数标为 1.24，摘要和正文的 1.48–3.58 范围不应理解为所有曲线均落入该范围。正文多节点 rollout 使用 GLM-4.6；补充组件消融使用 GLM-4.5-fp8，不能仅因数值近似就视为完全相同实验。原文的“一个 H100 节点”通常是包含 8 张 H100 的节点，部分表题简写为两个 H100 节点；应结合部署段理解，不能读成总共两张卡。框架采用情况是本次 arXiv 版本中的报告，译文保留源码链接与语态。